课程笔记

软件工程的来龙去脉

南京大学《生成式软件工程》第 05 讲：从软件危机、Royce 瀑布模型到 LLM 时代的 Intent–Spec–Impl Gap

lecture-to-notes

2026 年



视频作者/频道：Bilibili：绿导师原谅你了（南京大学生成式软件工程 2026）
发布日期：2026 年（第 05 讲录播）
视频时长：99 分 59 秒
视频链接：https://www.bilibili.com/video/BV1Nyeq6qEt8

目录

1 开课：当模型学会实时决策 **3**

1.1 本周新闻：实时决策模型打开的新赛道 3

1.2 工作性质在改变：90%、taste 与可验证边界 4

1.3 The Bitter Lesson 与数据分布工程 5

1.4 把更多智力活动变成可训练对象 6

1.5 本章小结 7

2 回顾：软件是 Intent、Spec、Impl 的组合体 **7**

2.1 本章小结 8

3 今天的答案：直接做一个 MVP **8**

3.1 MVP 的收益逻辑 8

3.2 案例：把设备抽象成干净的接口 8

3.3 可验收的结果：让 agent 自己收敛 9

3.4 从 app 到 API：软件公司的消亡 10

3.5 本章小结 10

4 为什么要回头看历史 **11**

4.1 讲者的立场：智慧没有消失 11

4.2 回到 1960 年代：昂贵的码农 11

4.3 1968 年的“明日学校”：需求的想象力早就到位了 12

4.4 本章小结 13

5 软件危机与工程化：从 Dijkstra 到 Apollo **13**

5.1 软件工程与其他工程的根本区别 13

5.2 Software Crisis：三件事一起失控 14

5.3 Dijkstra：从 goto 到“谦逊的程序员” 14

5.4 1968 年的经济论证与六个技术论据 16

5.5 Apollo 与 Margaret Hamilton：把不可能当成可能 16

5.6 软件工程的诞生：从英雄到 checklist 17

5.7 本章小结 18

6 Royce 与瀑布模型：被误读最严重的一篇经典 **18**

6.1 一篇回答“如何交付大型软件”的论文 18

6.2 为什么“瀑布假设”必然昂贵 19

6.3 Royce 的五条建议 20

6.4 误读链：从教科书到采购制度 21

6.5 为什么这个误读活得这么久 22

6.6 本章小结 23

7 用数据依赖图重新理解瀑布、MVP 与 TDD **23**

7.1 Testing：第一次“被经历”而非“被分析” 23

7.2 把瀑布模型看成一张数据依赖图 24

7.3 Do It Twice、MVP、敏捷与 TDD 的关系 . . . 25
7.4 阅读经典与一手信息 . . . 26
7.5 本章小结 . . . 27

8 软件工程的方法和工具：为每个阶段造一门语言 27
8.1 不同阶段，不同的“语言” . . . 27
8.2 checklist 为什么不是 nonsense . . . 28
8.3 需求与测试：把规则变得可检查 . . . 28
8.4 维护：改一句需求会牵动什么 . . . 29
8.5 谁来完成这些工作 . . . 30
8.6 本章小结 . . . 30

9 把契约写进语言：从 Eiffel 到 Dafny 与 Verus 30
9.1 Design by Contract：调用者与实现者各自负责什么 . . . 30
9.2 Eiffel：把契约写进语法 . . . 32
9.3 今天的活跃方向与一个可检验的预测 . . . 33
9.4 本章小结 . . . 33

10 UML：用语言的方法解决 Intent–Spec–Impl Gap 33
10.1 为什么需要中间语言 . . . 33
10.2 阅读 796 页的规范 . . . 34
10.3 判断与边界：UML 死了，问题没有 . . . 35
10.4 本章小结 . . . 35

11 总结与延伸 36
11.1 讲者的收束 . . . 36
11.2 本讲义对全课主线的压缩 . . . 36
11.3 边界与开放问题 . . . 36
11.4 给你的三个可执行动作 . . . 36
11.5 下一步要问的问题 . . . 37

1 开课：当模型学会实时决策

这门课上一讲处理的是版本管理，从这一讲开始进入软件工程本身。讲者没有直接跳到方法论，而是先用大约二十分钟处理了一个前提问题：如果模型已经能写代码、甚至能在毫秒级做决策，那么今天学软件工程究竟是为了什么。这一节整理讲者对近期技术进展的判断，它们是后面所有历史讨论的动机。

1.1 本周新闻：实时决策模型打开的新赛道

讲者用一条本周新闻开场：TypeSafe AI 发布了可以实时玩 DOOM 的模型。他根据公开材料推测，这大概是一个 prefill-only 的 transformer，一次并行输出全部概率，不需要逐 token 自回归生成；输入是把游戏状态序列化成的 JSON，输出是一个做完决策的 token。这个 token 不说人话，但能被游戏消费，因此可以直接做事。讲者把它放在大语言模型与过去那种很小的模型（例如决策树）之间，称它为一个通用的、可训练、可真正干活的中间形态。这条新闻最有价值的部分不是模型本身，而是它把延迟压到了一个临界点：毫秒级到百毫秒级，也就是准人类的实时决策。



图 1: 讲者用于开场的本周新闻页：TypeSafe AI 的实时决策模型与 System 1 定位，页面同时预告了后续“AI 进化速度”的内容。[1]

延迟下降会改变什么，讲者给了两个具体场景。第一是数字人：现在的 AI 数字人与人对话有明显延迟，因此缺少“人感”；如果有 100 毫秒、乐观一点 50 毫秒的决策模型，就可以用它按对方说的话实时生成脸上的微表情。第二是陪伴类交互，例如哄小朋友睡觉。这两个例子的共同点是：能力原本就存在，只是被延迟卡住；一旦延迟进入人的感知阈值以内，产品形态就会变。讲者据此认为，这条新闻“把赛道打开了”，而且它不是全新的技术，只是过去没有人把它推到公众面前。

[1] 视频画面时间区间：00:01:37–00:03:19。若为裁剪图，时间区间仍对应原视频画面。

从 1956 到 2026：讲者展示的一条时间线

课堂上播放了一段由 GPT-6 生成的 AI 发展史视频，时间线大致是：1956 到 1960 年，感知与规划机器走出实验室；1970 到 1990 年进入人工智能寒冬；1997 年搜索算法第一次击败人类世界冠军；2012 年 AlexNet 之后变化开始加速；2016 年展示出新的可能性；2017 年 transformer 与 BERT 出现；2020 年训出 GPT-3，当时很多人不理解为什么要做那么大的参数模型；2022 年 GPT-3 的路线被理解；2024 年 Sora 发布，同一年的 9 月出现 Chain of Thought 的 o1 模型，随后第一个开源 CoT 模型 DeepSeek-R1 出现。讲者的提醒是：这条线的斜率仍在变陡，“就业市场上面临的挑战”与“从此打开的机会”是同一件事的两面。

1.2 工作性质在改变：90%、taste 与可验证边界

第二条新闻是 OpenAI 的 Noam Brown 访谈。讲者复述了其中三个判断。第一个是关于工作量的重新分配：如果 AI 能做一个人 90% 的工作，那个人的注意力就会集中到 AI 做不好的 10% 上。第二个是关于这 10% 的性质：它主要是 taste，也就是“对接下来该做什么”的直觉；Noam Brown 预计一两代模型之后，模型的 taste 就会比他强。第三个是关于边界的：哪些能力可验证、哪些不可验证。



图 2: Noam Brown 访谈页：90%/10% 的分工、taste 的含义，以及“可验证”与“不可验证”的分界线。[2]

讲者对 taste 的解释很具体。当前模型被训练成了 prompt 的“舔狗”，你指向哪里它就去哪里；同时它有 reward hacking 的倾向，会用最快最脏的方式把任务做完，因为在强化学习里“完成任务”才有奖励。于是它执着地去做立即有反馈的事，产出在长期主义视角下未必令人满意。讲者认为这里并不是无解的：可以用更多合成数据去告诉模型“你可以想得更远一点”，因为 chain of thought 的头几个 token 就决定了它要做什么。换个说法，taste 可以被部分地工程化，只要有人愿意为“更远”这件事构造反馈。

[2]视频画面时间区间：00:08:09–00:09:17。若为裁剪图，时间区间仍对应原视频画面。

关于可验证边界，讲者的观察比新闻本身更细。品味看起来不可验证，但也许没有那么难；数学看起来可验证，其实也没有那么容易。他举了一个笑话：有人声称证明了某个千禧年难题，最后大家发现他对问题的形式化定义本身是“nothing”，于是他把几千页证明的结论落到了一等于一。这个笑话的意义在于，即使问题被形式化，从自然语言描述的题目到形式语言描述的题目之间仍然有 gap，而 gap 里的错误不会被证明器发现。这条边界在后面的软件工程讨论中会反复出现：能被机器检查的东西是有限的，剩下的是人对意图的把握。

1.3 The Bitter Lesson 与数据分布工程

第三条新闻是 OpenAI 的 all in RSI（recursive self-improvement，递归自我改进）。讲者由此回到 Sutton 的 The Bitter Lesson：scaling law 到今天还没有停止，而这一轮的变量并不只是数据量。他给这个变量起了一个名字：数据分布工程。



图 3: The Bitter Lesson 与“数据分布工程”的飞轮：学什么与何时学、怎样学、持续调整，以及 DoReMi 实验给出的证据。[3]

讲者的推论从 GPT-6 的空间推理能力出发：模型能理解图片里的空间关系，在画图和生成上都表现更好，这几乎必然来自训练时加入了这部分数据，而且不只是基座模型的预训练，后训练与强化学习阶段也有对应的反馈信号。这解释了为什么过去要靠“买书烧书”取得绝版数据（他提到 Anthropic 买入书籍），而现在可以在强化学习阶段直接给基座模型注入一个新领域，让它在这个领域上被训练，结果是所有任务一起变好，这一点“非常恐怖”。

更让讲者意外的是数据量的方向。他观察到模型的训练脉络大致是语言、数学、多模态、空间感知；每增加一个新的人类领域，所需要的数据不是更多，反而更少。原因是零起点训练空间任务需要大量模拟器数据，而 reward hacking 会让模型抓住渲染中极细微的像素差别，学不到泛化；反过来，一个既会语言又会数学、还会写程序的基座模型，再加少量空间数据就能泛化。人类的学习顺序恰好相反：婴儿先与世界交互建立世界模型，一两年之后才学会说话，再学数学，最后才是研究品味。但讲者强调，无论按什么顺序训练，只要数据足够多、有 scaling law，模型就能训成。

[3] 视频画面时间区间：00:10:41–00:15:23。若为裁剪图，时间区间仍对应原视频画面。

数据分布工程：讲者命名的这一轮 scaling law

飞轮的四个要素是：（1）学什么、何时学，即在什么阶段接触哪些经验；（2）怎样学，即数据的比例、难度、顺序与训练方式；（3）持续调整，即根据学习进展、跨域迁移与泛化表现改变训练分布；（4）验证机制，例如 DoReMi 实验：某个领域的训练占比降低之后，该领域的困惑度仍然可以改善。讲者的判断是：只要训练不崩、训练与推理保持一致，架构与算法的重要性相对下降，谁掌握更大规模的数据分布工程能力，谁就能继续变强。

这一判断解释了讲者对学术界的担心。一方面投稿量到达新的巅峰，其中不少是“AI slop”；另一方面工业界不再关心小规模实验，因为分布工程的能力不在学术机构手里；一个在小规模上很酷的想法，今天也许只值得写成 blog post，因为对方的 RSI 闭环读到之后可以直接在真实数据上试。

1.4 把更多智力活动变成可训练对象

第四条新闻是“增加可训练领域”：把更多智力活动变成可以规模化训练的对象。幻灯片给了三组例子，每组都自带构造任务与验证结果的手段。

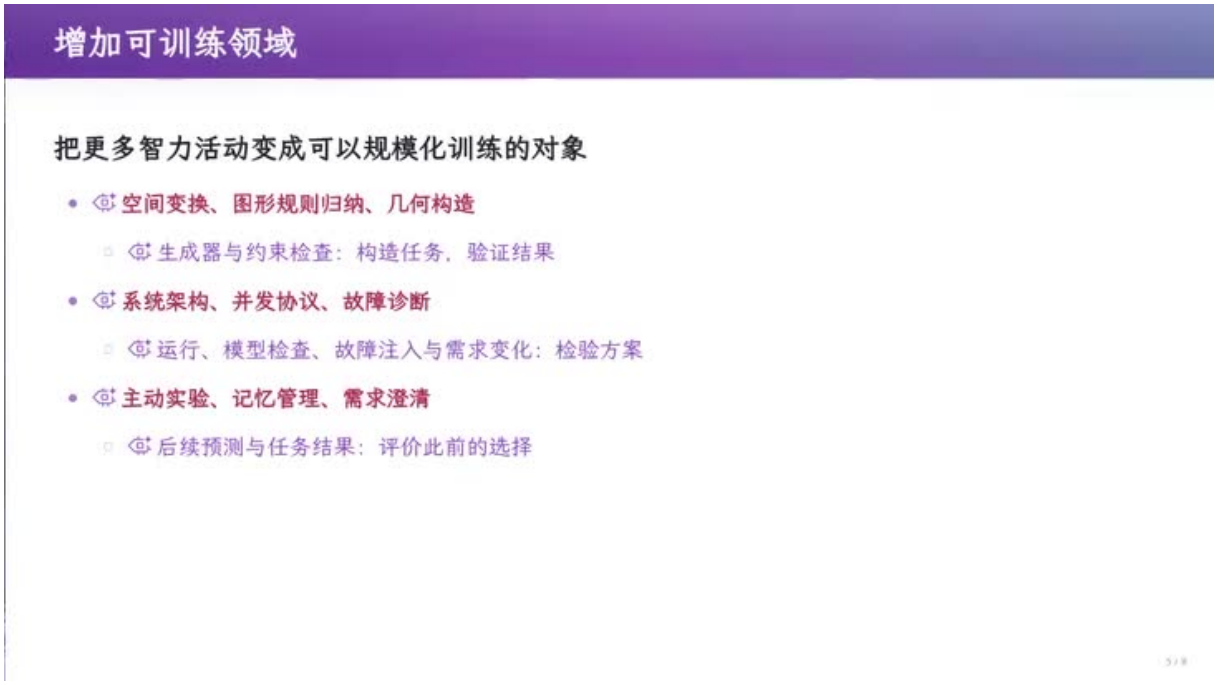


图 4: 增加可训练领域：三组“任务构造 + 结果验证”的配对，把过去被认为依赖天赋的活动变成可训练对象。[4]

第一组是空间变换、图形规则归纳与几何构造，用生成器构造任务、用约束检查验证结果。第二组是系统架构、并发协议与故障诊断，用运行、模型检查、故障注入与需求变化来检验方案。第三组是主动实验、记忆管理与需求澄清，用后续预测与任务结果来评价此前的选择。讲者特别提到，下一个实验会要求学生把自己的系统与一个物理设备连起来，哪怕只是花 200 块钱买一个小设备；而系统设计这种过去被认为是“架构品味”的能力，也许在这个学期结束之前就会被加进模型里；到那时他在课上嘲笑 AI 生成 slop code 架构糟糕、一加需求就崩溃的段子，可能就永远不好笑了。

[4]视频画面时间区间：00:16:58–00:17:28。若为裁剪图，时间区间仍对应原视频画面。

讲者把这条线索接回 RSI：一旦“找新领域、建环境、建验证器、优化数据分布”这个闭环接近收敛，AI 就可以自己去发现下一个领域。他听到的消息是 Anthropic 要建 wet lab 做生物工程实验；也可能是预测老鼠的行为、情感计算或化学。其经济含义是：新建一个实验室、投入几十亿美元，模型能力就往上抬一个台阶，这是一条可预期的收益路径，而学术界“随机想一个 idea 然后投出去等 peer review”的体系会承受很大压力。讲者由此提出了一个不给答案的问题：在这个体系下，我们还能剩下什么。

他的收束不是焦虑，而是一条很具体的分工：人类是竞争社会，生存竞争会让“斩杀线”不断上移，每个人要考虑的是如何在被斩杀之前搭上最后一班车；当下能确定的只有一件事，能驾驭并能理解 AI 的人会活下来，所以还需要上课，还需要理解软件工程。

1.5 本章小结

这一节回答了“在模型已经会写代码、还会实时决策的时代，为什么还要学软件工程”。讲者的答案有两条：第一，模型能力的增量来自数据分布工程与 RSI 闭环，这个飞轮把越来越多的智力活动变成可训练对象，所以任何具体技能的半衰期都很短；第二，能被训练接管的部分之外，仍然剩下“判断接下来该做什么”和“把意图说清楚”这两件事，而软件工程史正是人在没有模型可用的情况下，为这两件事积累方法的四十年。下一节先回到上一讲留下的问题，看清这门课要处理的到底是什么东西。

2 回顾：软件是 Intent、Spec、Impl 的组合体

上一讲结束时留下了一个悬念：如果从零开始构造一个系统，版本管理只是第一步，接下来到底应该怎么构造。要回答它，先要把讨论对象重新定义清楚。



图 5: 复习页：软件被定义为 Intent、Spec、Impl 的组合体，版本管理是把它按文件系统快照保存下来的手段。[5]

[5] 视频画面时间区间：00:17:43–00:19:12。若为裁剪图，时间区间仍对应原视频画面。

本课程的基本框架

软件是 Intent（想要什么）、Spec（怎样组织）与 Impl（具体实现）的组合体。所谓软件开发，就是不断推出新的版本，直到满足客户需求；版本管理把这些版本以文件系统快照的形式存储下来，于是自然会遇到分叉与冲突，也就解释清楚了 SVN、Git、Jujutsu 这些工具的设计原因。

这个框架的关键在于三者之间存在 gap：Intent 与 Spec 之间、Spec 与 Impl 之间有天然的距离，而客户与开发者之间的信息差让这两个 gap 都无法一次说清。讲者用这个框架串起整门课：软件之所以成为一个世纪性难题，是因为意图必须经过两层翻译才能变成能运行的系统，而每一层翻译都会发生漂移。

2.1 本章小结

本节确认了后续讨论的对象：软件是 Intent、Spec、Impl 的组合体；版本管理解决的是“把已经产生的东西存下来并管理分歧”，而真正的困难是“怎么把意图一步步变成实现”。从空的项目（一个模糊的 Intent）出发，最自然的做法是什么？讲者给出的答案是先做一个 MVP，这正是下一节的主题。

3 今天的答案：直接做一个 MVP

在讲历史之前，讲者先给了一个属于今天的答案。它不是方法论名词，而是他自己在前一天晚上做出来的东西：一个会管理实验室里打印机和扫描仪的 self service 工具。这个案例把 MVP、抽象、agent 自我迭代三件事一次性演示完了。

3.1 MVP 的收益逻辑

MVP 是 minimum viable product，最小可行产品。讲者给的理由非常朴素：做得多就错得多，做得少就错得少，所以要做一个最小、但和最终产品最接近的东西。它的价值不在实现本身，而在于补 gap：因为第一个版本是 AI 生成的 slop code，实现随时可以扔掉；但通过它，你可以知道 Intent 哪里没有弄对、Spec（例如架构）还有没有改进空间，然后再做几个版本的迭代，最后才交付。换句话说，MVP 的作用是把“想清楚”这件事从会议室搬到一个可以运行、可以被观察的对象上。

MVP 在本课程框架里的位置

MVP 不是“先做一个粗糙的正式产品”，而是用可丢弃的实现去校验不可丢弃的两件事：Intent 与 Spec。讲者反复强调实现是可丢弃的（AI 生成的成本极低），因此迭代的边际成本主要花在“发现意图与设计的问题”上，这正是它比反复评审文档更有效的原因。

3.2 案例：把设备抽象成干净的接口

讲者的实验室里有一台标签打印机和一台扫描仪，每台设备都有自己的原厂软件，大家各自用各自的那一套，体验很差。他的做法是把这些软件全部屏蔽掉，重做一层抽象：扫描仪的接口是 scan（正面向上还是下面向上、单面还是双面、dpi 是多少），打印机的接口是 print(image)。接口确定之后，设计与实现就解耦了，软件层只工作在接口层上。讲者说这是非常典型的软件工程设计，也正

好对应操作系统课上的设备抽象：把一个设备抽象成 read、write、ioctl 的接口，只不过这里换成了一组更好用的接口。



图 6: 讲者的 agent 逆向后给出的设备通路分析：标签打印机与扫描仪各自直连 USB，在软件里是两条独立路径，驱动层级与权限规则都不相同。[6]

这个案例的技术细节很具体，也恰好说明“非标设备”有多麻烦。标签打印机在幻灯片上写作 EZSCAN/ZMIN GL70，USB ID 为 03eb:9638，内核走 usblp 驱动变成字符设备，应用层直接往 /dev/usb/lpN 写裸字节，因此需要一条 udev 规则把节点权限设成 0666，否则只有 root 能用；它使用 ZMIPL1 指令加一比特位图，不经过 CUPS。扫描仪是一台 Canon imageFORMULA R40，USB ID 为 1083:1679，走 libusb 直访，应用层用 SANE 的 canon_dr 后端通过 scanimage 读取，与打印机互不连接，各自以 USB 连到同一台 Linux 机器。讲者的 agent 把这两条通路分别分析清楚，再各自写一层接口，前端只需要“用嘴输出”需求即可。

3.3 可验收的结果：让 agent 自己收敛

最值得注意的是打印机那个“不知道是什么”的参数。这是一台热转印标签打印机，碳粉带在有像素的位置被加热，把碳粉烫到标签上；它有一个表示加热程度（图像深度）的参数，谁也不知道应该设多少。agent 的做法是一张一张地打标签：先打一张上面写着序号“15”的，然后问讲者是淡了还是浓了。讲者说，如果这时接一个摄像头对着打印机，就可以连人都不需要，agent 自己一张一张地打、自己判断，完全自动地收敛到这个参数。它最初试图走系统的 lp 驱动路径，后来发现可以直接用一个 PCL 驱动做通用打印。

讲者对过程的描述是“我完全没有监管，我只是问他要一个结果，是一个可验收的结果”。这是他反复强调的 agent 协作方式：不去控制中间步骤，只把验收标准定义清楚，让 agent 一轮一轮地把事情做完。这与前文 RSI 的讨论是同一个结构：有可验证的结果，才有一轮一轮的自我改进。

[6] 视频画面时间区间：00:23:47–00:24:43。若为裁剪图，时间区间仍对应原视频画面。

3.4 从 app 到 API：软件公司的消亡

讲者用豆包手机把这个逻辑推到产业层面。讲者的判断是：这些 app 都不重要了，如果有一个又快又足够聪明的决策模型，app 就沦为“点外卖的接口”“查询数据的接口”，只需要提供 API 给 agent 用，而不需要一个给人用的界面。他认为豆包手机是很好的工程产品，因为它体现了同样的软件工程智慧：先把能力抽象成接口，再赌模型会快到来得及替人点手机。他也顺带评价了另一个方向的产品：把旗下所有服务整合到一个 agent 应用里，思路是对的，但模型的速度与算力还不成熟，所以看起来像未来的半成品。

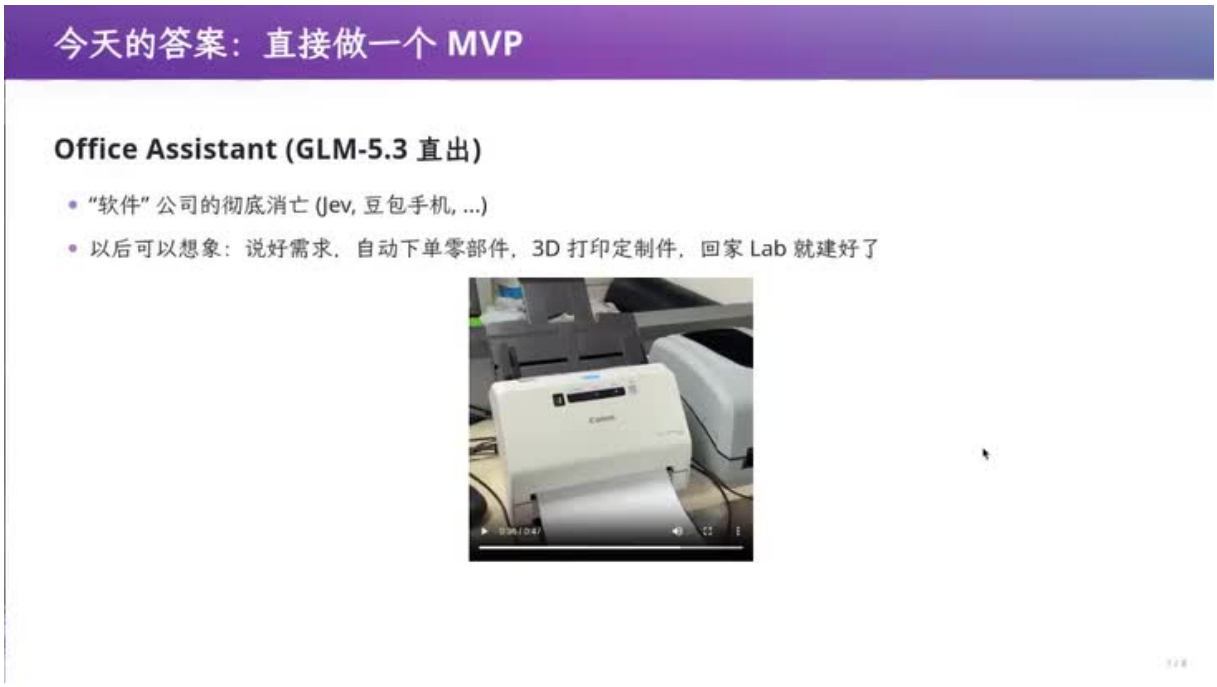


图 7: 本节的结论页：AI 直出的 OfficeAssistant 与“软件公司的彻底消亡”，以及讲者对“说好需求就自动造物”的展望。[7]

讲者给的产业推演是：一台标签打印机卖几百块钱，硬件靠成熟供应链，过去软件成本极高，因为要做 Windows 驱动、还要做 macOS 驱动，否则用户会抱怨；今天这部分成本被 AI 抹掉了，而且软件还能按个人需求定制，不再需要追求通用。他由此预测依赖“软件 + 驱动 + 外包”的软件公司会彻底消亡，涉及多少外包公司与产业链上的人。这一段是讲者的判断与展望，视频里没有给出市场数据支撑，读者应当把它当作一个有明确推理链条的预测，而不是已发生的事实。

3.5 本章小结

本节给出了今天可用的答案：直接做一个 MVP。操作上有三条要点：第一，用它来补 Intent 与 Spec 的 gap，实现是可丢弃的；第二，把设备、服务抽象成干净的接口，让设计与实现解耦；第三，把验收标准定义清楚，让 agent 自己一轮一轮收敛。这套做法已经能覆盖过去需要厂商软件和外包团队的工作，因此真正值得回头问的是：这些方法从哪来、它们此前解决的是什么问题。下一节开始讲历史。

[7] 视频画面时间区间：00:25:18–00:27:05。若为裁剪图，时间区间仍对应原视频画面。

4 为什么要回头看历史

讲者先把今天的答案说清楚，然后才进入历史。这样做有一个明确的目的：他并不认为软件工程是一堆过时的仪式，而是要说明这套东西是在什么约束下被发明出来的，以及今天哪些约束已经变了、哪些没有变。

4.1 讲者的立场：智慧没有消失

讲者的观察是：我们这一代人已经生在“软件死掉了”的时代，软件工程的很多前提确实变了，但软件工程的智慧没有消失。他的建议是透过历史去理解“构造软件”这件事中大家发现、发明了什么，再用这些理念指导今天与 agent 的协作，从而成为一个更好的 agent-native 软件开发者。换言之，历史在这里不是怀旧，而是判据的来源：只有当你知道某条方法是为了对付哪个具体困难而出现的，你才能判断它今天是否仍然必要。

4.2 回到 1960 年代：昂贵的码农

第一站是 1960 年代，软件还是一个新生事物。讲者给这张幻灯片列了三条前提。

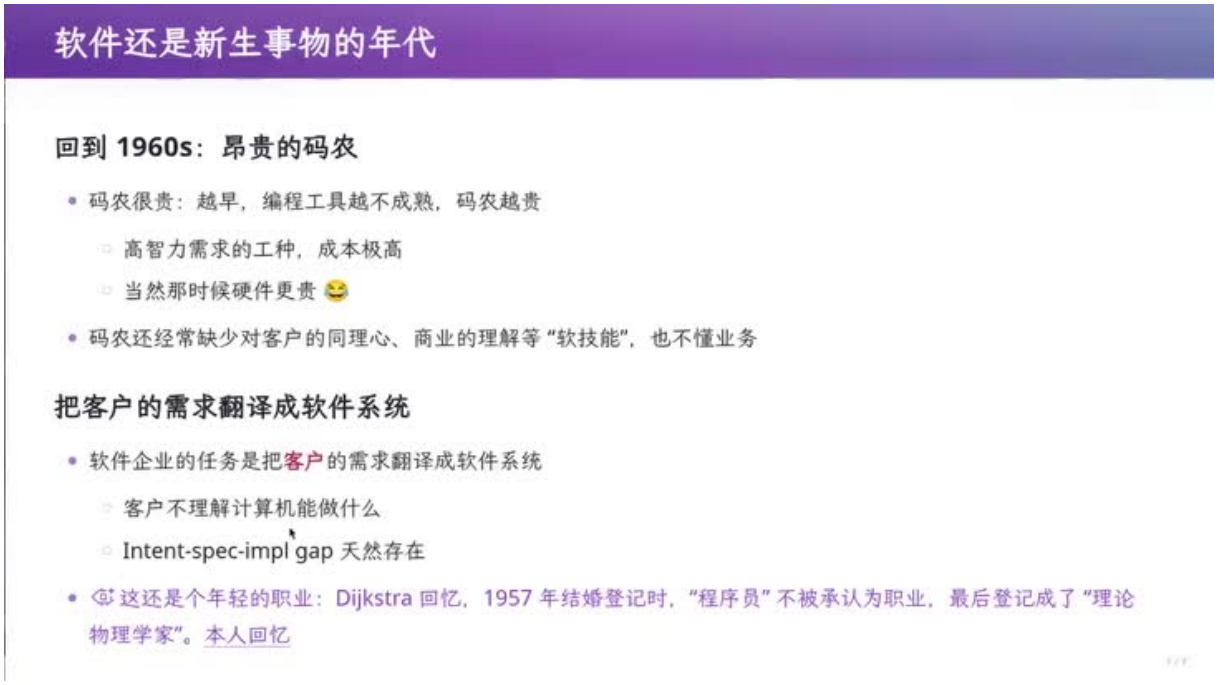


图 8: 1960 年代的软件生产条件：码农很贵、硬件更贵、客户不理解计算机能做什么，而三者共同制造了 Intent 与实现之间的天然 gap。[8]

第一条是码农很贵，而且越早越贵：编程工具越不成熟，对人的智力要求就越高，成本也越高。讲者用了一个类比：那时候写机器码如果不画流程图，就跟今天让模型直接写汇编一样，模型能只看二进制序列写出汇编，人做不到，除非经过过度的预训练。第二条是硬件更贵：一台计算机要占很大的房间，作为最先进的设备制造成本极高；讲者的说法是，早期大家还不知道怎么造计算机，不敢偷工减料，成本自然高。第三条是客户与开发者之间的信息差：客户也不知道计算机能做什么，例如 DARPA 的官员、要开发飞控系统的团队，他们给出的是一句“把人类送上月球”这样的需求。

[8] 视频画面时间区间：00:30:54–00:34:28。若为裁剪图，时间区间仍对应原视频画面。

这张幻灯片还写了一条容易被忽略的限制：程序员当时还算不上一个被承认的职业。讲者举的例子是，1957 年登记结婚要填写职业时，程序员不被承认为一种职业，最后只能登记成“理论物理学家”。同页还提到，这些经过层层筛选、数学训练很多的人，往往缺少对客户的同理心与商业理解等软技能；他们可能只想要造世界上最好的操作系统，但软件企业必须承担把 Intent、Spec 与 Impl 之间的 gap 补上的职责。这就是公司存在的位置：客户拿出“1000 万美元开发一个操作系统”这样的天文数字，公司负责把它变成能交付的系统。

为什么“昂贵的码农”决定了那个时代的工程学

当人是最贵的资源时，一切方法都围绕“减少人力的浪费”展开：画流程图是替人记账，写文档是把人的判断固化下来，评审是用流程替代个人记忆。理解这一点，才能理解后面 Royce 为什么把“让设计师在分析阶段抢资源”这种组织细节写进一篇技术论文里：在 1970 年，组织与流程本身就是技术。

4.3 1968 年的“明日学校”：需求的想象力早就到位了

第二站是一个有意思的历史材料。1968 年 9 月号的《Boys' Life》杂志畅想了计算机化的学校：终端即时判分、针对薄弱环节辅导、让学生按自己的节奏学习。讲者说，这些功能到今天都没有完全实现，我们现在还在上每节 50 分钟的课，而这份杂志的封面与原刊已经写清了“因材施教”这个意图。

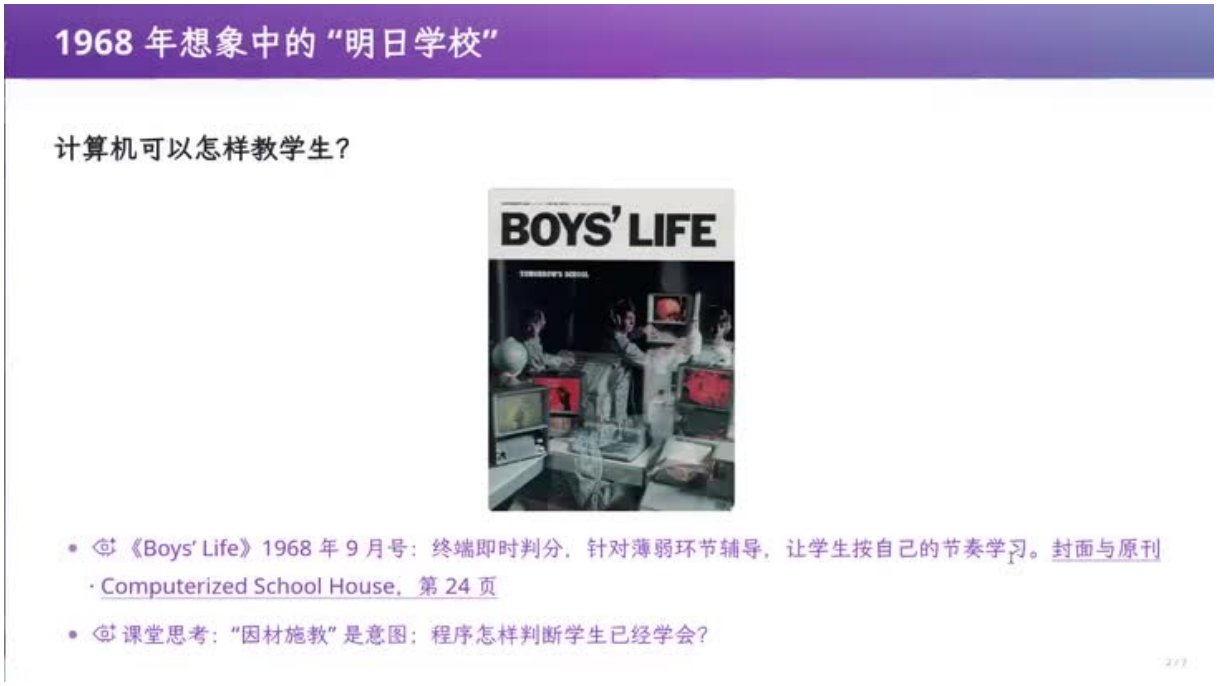


图 9: 1968 年《Boys' Life》对“明日学校”的想象，以及讲者留下的课堂思考：“因材施教”是意图，程序怎样判断学生已经学会？[9]

幻灯片末尾留了一个问题，正好是这门课的主题：因材施教是意图，但程序怎么判断学生已经学会？这个问题把“需求”与“可执行的判定”之间的距离暴露出来了。讲者用它来说明当时的 gap 有多大：客户脑子里是 1968 年杂志上的那种想象，而程序员脑子里是汇编和 goto。讲者还提到他

[9] 视频画面时间区间：00:35:24–00:36:02。若为裁剪图，时间区间仍对应原视频画面。

的幻灯片是用 AI Copilot 生成的，方括号里的内容是他自己写给 AI 的话，AI 会把它们近乎逐字地保留下来，其余部分由 AI 发挥——这也解释了这张历史材料是从哪里找来的。

4.4 本章小结

本节给出了读历史的理由，并交代了起点：软件的意图从一开始就超出当时的技术能力，而生产力、客户认知与职业地位三方面的约束，共同制造了巨大的 Intent 与 Impl 之间的距离。下一节看这个距离如何演变成一场危机，以及人们是怎么为它建立工程化的应对方式的。

5 软件危机与工程化：从 Dijkstra 到 Apollo

1960 年代那三条约束把软件推到了失控的边缘，本节前半部分讲这场危机与 Dijkstra 的两篇名文，后半部分讲 Apollo 与 Margaret Hamilton 如何给出了“把软件当作工程”的第一个样板。

5.1 软件工程与其他工程的根本区别

讲者用一页幻灯片说明软件为什么不能照搬其他工程的经验。它先指出：物理世界的需求只能适应、不能改造，材料强度在出厂时就确定了，楼层高度与容纳人数由自然规律决定，所以桥梁与炼钢的演化一直是增量的；而软件的需求是高度定制化、由人拍脑袋想出来的，因此一直被认为高度依赖人类智慧，操作系统与编译器这样的软件也需要多年专业积累。

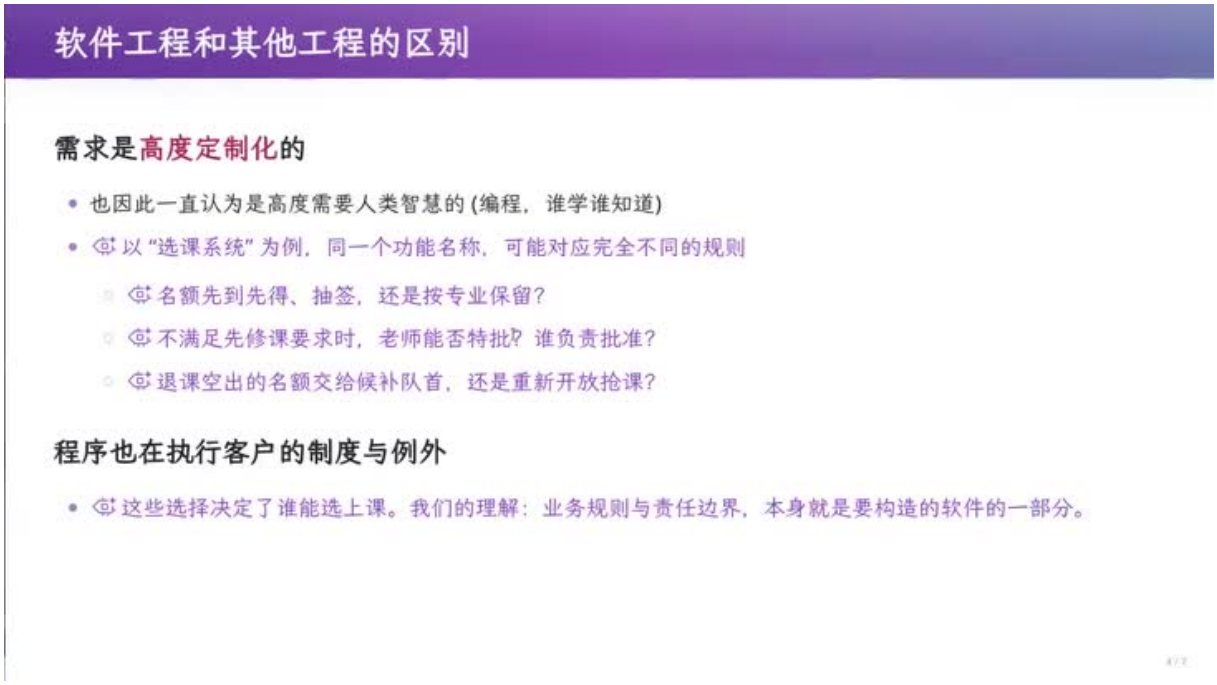


图 10: 以选课系统为例说明需求的高度定制化：同一个功能名称背后可能对应完全不同的业务规则与责任边界。[10]

同一页幻灯片把这一点落到一个具体的例子上：选课系统。同一个功能名称，可能对应完全不同的规则。名额是先到先得、抽签，还是按专业保留？不满足先修课要求时，老师能否特批，谁负责批准？退课空出来的名额是交给候补队首，还是重新开放抢课？讲者的总结是：程序在执行客户

[10] 视频画面时间区间：00:37:24–00:38:53。若为裁剪图，时间区间仍对应原视频画面。

的制度与例外，因此业务规则与责任边界本身就是软件的一部分。他还补了一句“毕业前最后一秒，所有的口子学校都能给你开”，说明真实业务里总有例外，而例外必须被写进需求，否则实现只能靠猜。

与其他工程的根本区别

其他工程的需求由物理世界给定，工程师只能在既有约束内做增量改进；软件的需求由人给定，包含制度、例外与责任归属，既可以推倒重来，也容易在细节上自相矛盾。所以软件工程的困难不在“造不出”，而在“说不清”：同一句话在不同的人那里对应不同的规则集合。

5.2 Software Crisis：三件事一起失控

环境与需求确定之后，问题就显形了。幻灯片把它概括为进度、成本、质量一起失控：软件开发进度无法预测，开发成本崩溃式超支，质量与维护也都是问题。讲者补充说，这个时期计算机科学家已经在观察软件中有趣的现象，例如后文要讲的 goto 问题，而这些观察正是软件工程这门学科的起点。

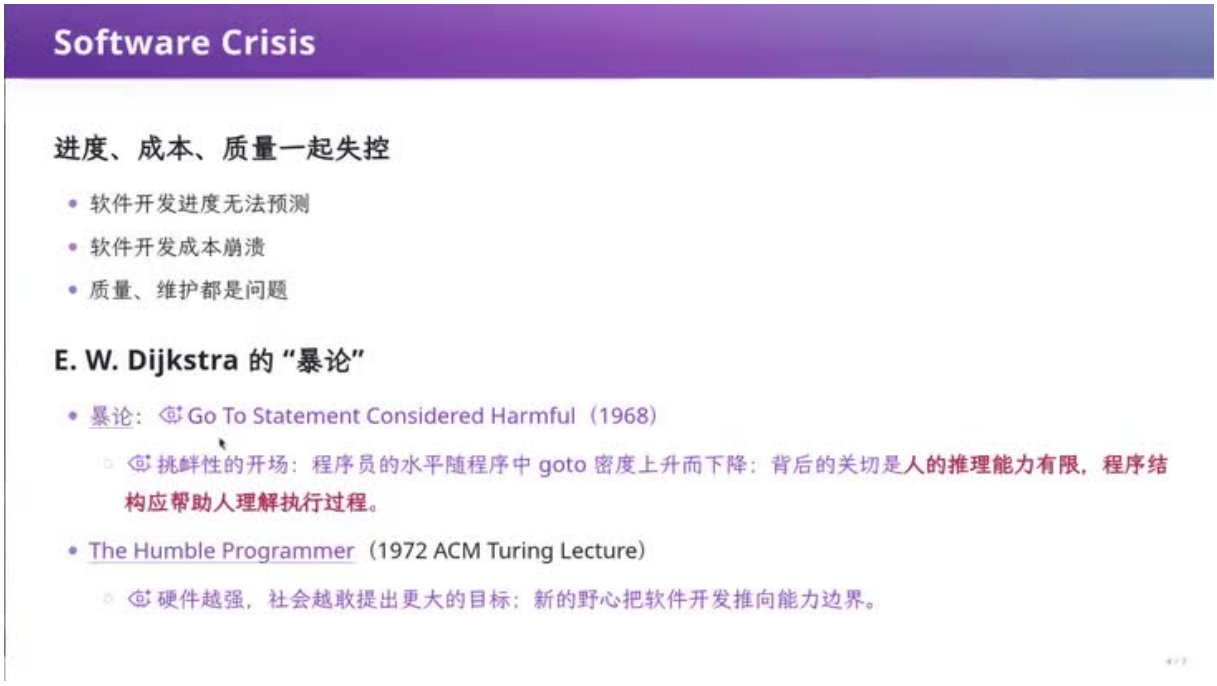


图 11: Software Crisis：进度、成本、质量一起失控，以及 Dijkstra 1968 与 1972 年两篇文章的定位。[11]

5.3 Dijkstra：从 goto 到“谦逊的程序员”

第一篇文章是 1968 年的《Go To Statement Considered Harmful》，发表在 CACM 11(3)，讲者称它是计算机科学史上最著名的短文之一。它的断言来自作者多年的观察：程序员的水平与所写程序中 goto 语句的密度呈反比。讲者提醒，这个说法本身是一句“暴论”，但它背后有实实在在的论证。

[11] 视频画面时间区间：00:39:41–00:40:34。若为裁剪图，时间区间仍对应原视频画面。



图 12: 讲者用 AI 阅读助手得到的 Dijkstra 短文摘要：静态文本与动态过程之分，以及刻画过程推进所需的文本索引与动态索引。[12]

论证的核心是一组区分。程序员真正的工作对象不是静态的程序文本，而是程序控制下随时间展开的执行过程，最终要满足规格的是这个动态过程；而人的智力擅长把握静态关系，弱于想象随时间演化的过程，因此推理的桥梁必须尽可能平凡。为了给过程的推进建立坐标系，Dijkstra 分析了几种情况：纯赋值语句序列只需要一个文本索引；加入条件、选择、case 与条件表达式之后，单个文本索引仍然够用；加入过程调用之后，还需要知道是“哪一次调用”，于是要用一串文本索引，长度等于调用深度；加入循环之后，文本索引不再充分，需要为每次进入循环配一个动态索引，也就是循环计数。结论是：过程推进总可以由文本索引与动态索引的混合序列唯一刻画，而这些索引的值不受程序员控制、由书写方式或运行时产生，因而是独立坐标。goto 之所以危险，正是因为它让这种坐标变得难以描述。

讲者随后补了一个反例，避免了把这条结论教条化：Linux 内核里有大量 goto，它常被用作一种正确的机制——在函数里申请的所有资源都在一个 release 段里逐个检查释放，任何中途退出的路径都跳到那里，程序就基本正确了。所以“goto 有害”讨论的不是某个关键字，而是“人能不能读、能不能维护”：只有人能读，软件才可能维护得下去。第二篇文章是 1972 年的图灵奖演讲《The Humble Programmer》。讲者说，如果永远写 Online Judge 的小程序，就不会有软件危机，因为程序一旦调对就再也不需要维护；而机器变强之后，社会敢提出更大的目标，软件要承担的意图越来越多，不确定性随之增加，系统也越来越复杂——这才是软件危机的来源。文章里还有两条今天读来仍然有效的判断：一是 nicely factored solutions 这种良好分解的思路，二是保守力量主要来自教育机构与舍不得重写旧程序的用户。

同一时期的材料还交代了这个职业的起点。幻灯片记录：1952 年 Dijkstra 成为荷兰第一位“程序员”，当时他在理论物理与编程之间摇摆，上司 van Wijngaarden 劝他：计算机必将长存，何不由你来把编程建设成一门受人尊敬的学科。讲者把这句话和 1957 年那栏“理论物理学家”放在一起看，说明职业身份的建立本身就是这段历史的一部分。

[12] 视频画面时间区间：00:40:54–00:41:15。若为裁剪图，时间区间仍对应原视频画面。

5.4 1968 年的经济论证与六个技术论据

软件工程的名字需要一个成立的时刻，讲者给出的是一组条件论证。第一是经济需要足够强：硬件价格预计再降 10 倍，软件成本必须降一个数量级，否则整个行业的账算不过来。第二是技术上可行，他列了 6 个论据。



图 13: 经济需要与技术可行的六个论据，以及阻力与结语：“像躲瘟疫一样躲开聪明技巧”，以“非常谦逊的程序员”姿态行事。[13]

六个论据可以压缩成三组。第一组关于解空间：只需考虑智力上可管理的程序，候选方案就大大简化；一旦限定到这个子集，就一次性大幅缩减了解空间。第二组关于工具与证明：正确性证明可以与程序同步生长，证明反过来成为设计启发；他还点出一个至今有效的判断：测试能有效地证明 bug 存在，却无法证明 bug 不存在。第三组关于抽象与分解：借助抽象，理解程序所需的智力投入不必随长度平方增长，甚至可以线性，文中还点出了若干关键的“抽象模式”；工具会反过来塑造思维，语言决定我们能思考什么，而脑力是最稀缺的资源，因此要“像躲瘟疫一样躲开聪明技巧”，追求系统而谦逊的语言；最后，层次化与良好分解是唯一能让人满意地解决复杂系统的途径。幻灯片还列出了阻力：设备费用会绑架校园选型，方法论教育会放大能力差距，在追求均质化的社会里未必受欢迎。结论是：唯有充分认识任务之艰巨、坚持朴素优雅的语言、尊重人类心智的固有局限，以“非常谦逊的程序员”的姿态行事，编程才能做好。

讲者提到，1968 年还有一场 NATO 软件工程会议，会议纪要可以直接读到；也差不多在同一个时期，还有一件更有象征意义的事。

5.5 Apollo 与 Margaret Hamilton：把不可能当成可能

接下来这个案例是 Margaret Hamilton 与她的团队，也是“把软件当作工程”最有象征意义的一次实践。幻灯片记录：她在 1964 年加入 MIT 仪器实验室（今 Draper 实验室），参与的是阿波罗制导计算机（AGC，Apollo Guidance Computer）的飞行软件；她先写无人任务中止时的代码，因

[13] 视频画面时间区间：00:43:07–00:43:27。若为裁剪图，时间区间仍对应原视频画面。

为认为它不会被触发，给程序取名叫 Forget It，结果真派上了用场；随后她晋升到“载人级”软件，这类软件必须能侦测并补偿硬件故障或宇航员的误操作。正是她提出了 software engineering 这个说法，并推动软件工程观念获得承认。



图 14: Margaret Hamilton 与 Apollo 软件：从 Forget It 到载人级软件，以及 Apollo 11 下降时 1201/1202 警报的处理。[14]

first principle：never going to happen can happen

讲者把这段历史的第一性原理概括为一句话：绝不发生的事也会发生。她要求把每一个“不可能”的假设都当成可能：假设硬件不可靠，假设程序会崩，但系统仍然要能工作。幻灯片给出的证据是 Apollo 11 登月下降时计算机过载，出现 1201/1202 警报；因为软件里有优先级与恢复机制，关键的下降控制任务得以继续，而 MIT 团队事先充分测试过恢复能力，地面团队才有依据判断可以继续登月。讲者据此总结了这件工程工作的三个动作：决定必须保住什么、实现恢复机制、验证它确实有效。

讲者用这段历史顺手打了一个今天仍然常见的思维定势：做 Online Judge 时，你假设题目是对的、假设题目一定能做出来；当你反复强化时，就 overfit 了这些隐藏假设。而软件面对的是一个开放世界：题目可能是错的，条件随时可能变化，甚至可能变成另外一个题目。把“绝不可能”纳入规格，正是软件工程与刷题思维的分界线。

5.6 软件工程的诞生：从英雄到 checklist

最后一个站点是“软件工程”作为一个学科被定义。幻灯片给的表述是：在成本、时间等约束下，把人的意图变成可工作的、可维护的、能够生存的软件系统；它覆盖的范围从大到团队组织管理，小到代码风格。

[14] 视频画面时间区间：00:47:14–00:47:34。若为裁剪图，时间区间仍对应原视频画面。



图 15: 软件工程的诞生与 IEEE 定义：系统化、有纪律、可量化的方法覆盖开发、运行与维护。[15]

IEEE 的定义强调三点：系统化、有纪律、可量化，覆盖软件的开发、运行与维护。讲者解释了这三点真正的意义：在 Hamilton 那个时代，一个项目需要一个好的 leader，换一个人项目就可能失败；而工程化之后，你可以拿一个 checklist 对着打勾，如果一、二、三、四全部满足，虽然不能保证软件成功，但失败的概率会小很多。也就是说，软件工程把“个人英雄主义”转换为可复制、可交接的纪律，而这份纪律是从数亿美元量级的教训中长出来的。

5.7 本章小结

本节回答的是“软件危机是怎么来的、人们用什么方式回应它”。软件的需求由人给定且充满制度与例外，因此进度、成本与质量会一起失控；回应它的方式，一是像 Dijkstra 那样把“人要能读懂过程”变成语言与结构的约束，二是像 Hamilton 那样把“绝不可能”变成必须被验证的工程动作，三是像 IEEE 的定义那样把个人英雄主义换成系统化、有纪律、可量化的流程。在这个背景下，1970 年出现了软件工程领域最著名、也被误读最严重的一篇文章，下一节专门讲它。

6 Royce 与瀑布模型：被误读最严重的一篇经典

这是本次课篇幅最长的一节，也是信息密度最高的一节。讲者带着学生读了原文，并且用了一整套自己的阅读工具（把论文转成 markdown、把插图嵌进去、在命令行里读），因此这一节的结论不是教科书里的“瀑布模型是什么”，而是“Royce 到底写了什么、为什么会被读反”。

6.1 一篇回答“如何交付大型软件”的论文

1970 年，软件工程刚刚诞生，既没有 minimum viable product 这个概念，也没有大语言模型；用讲者的话说，他所在的是 Pre-LLM 时代，任何流程改进都必须靠人和文档来完成。讲者把当年

[15] 视频画面时间区间：00:47:52–00:49:41。若为裁剪图，时间区间仍对应原视频画面。

的问题还原得非常朴素：从空的 repository 开始，怎么按时、不超预算地交付大型软件？Winston Royce 的论文《Managing the Development of Large Software Systems》就是对这个问题的回答，它画出了后来被称为“瀑布模型”的经典阶段图。

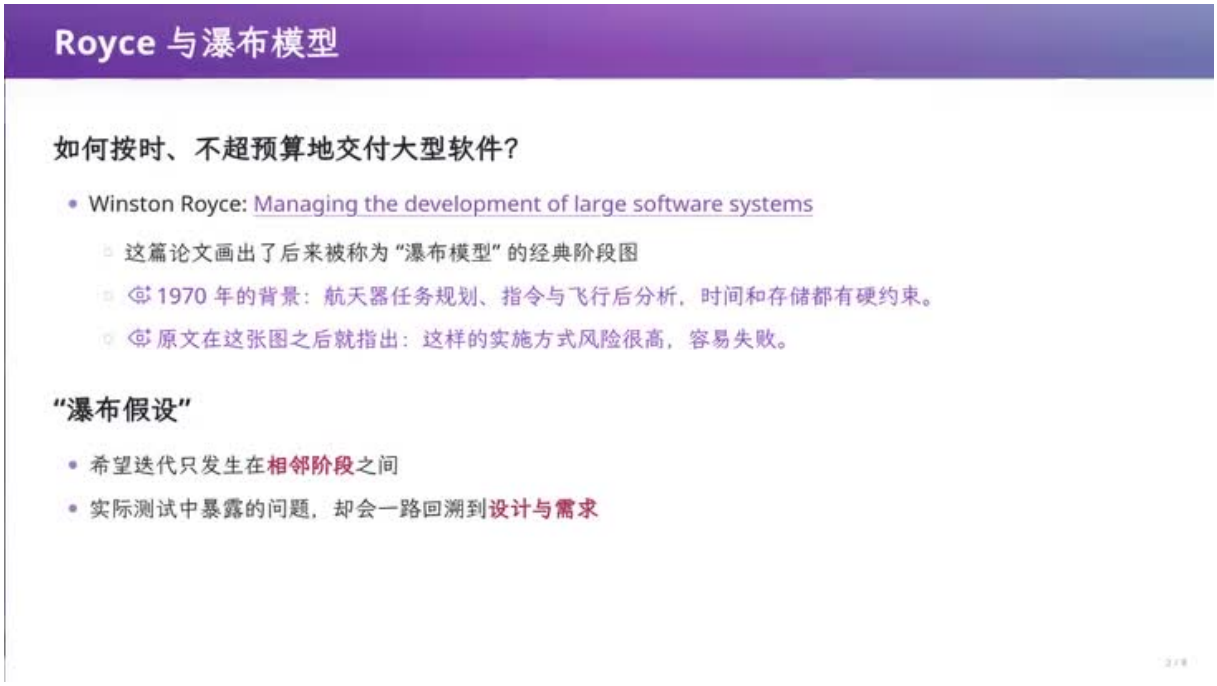


图 16: Royce 与瀑布模型：论文画出了经典阶段图，但原文紧接着就指出这种实施方式风险很高、容易失败；幻灯片由此提出“瀑布假设”。[16]

论文的背景是航天器的任务规划、指令与飞行后分析，时间和存储都有硬约束。讲者强调了两点。第一，这张著名的图不是 Royce 的推荐方案，图之后紧接着就写着这样的实施方式风险很高、容易失败。第二，所谓“瀑布假设”指的是希望迭代只发生在相邻阶段之间：需求只影响设计，设计只影响实现，实现只影响测试；而实际测试中暴露的问题，却会一路回溯到设计与需求。讲者因此说这篇文章是“被误读最严重的一篇”，学软件工程的人都在学瀑布模型，但原文恰恰是在批判它。

6.2 为什么“瀑布假设”必然昂贵

讲者从第一性原理给出了解释：测试是整个链条上第一个“物理事件”。在设计阶段，程序还不能运行，能够交付的只有文档，而文档没有强制约束——讲者用学生写实验文档的经验作比：你对实验文档的正确性有 guarantee 吗？最多是 best effort；代码改了、设计改了，文档不一致也就放在那里了。只有在测试时程序才真正跑起来，你才真正知道它的行为是什么。

一旦从 Intent 到 Spec 到 Impl 发生了漂移，而问题在测试阶段才被发现，就必须回头修改前面阶段。Royce 的判据是：缺陷被发现的时间越晚，返工的代价越大；而返工的代价取决于回溯距离，也就是从发现问题的地方往上追溯到必须修改的那个产物所经过的距离。测试位于链条末端，一旦它的错误出在需求上，回溯距离就是最长的那一段，于是超期或超支就发生了。

[16] 视频画面时间区间：00:51:07–00:51:47。若为裁剪图，时间区间仍对应原视频画面。

把“阶段顺序”当成纪律会放大成本

瀑布假设真正危险的地方不是“有阶段”，而是把阶段当成不可违反的纪律：既然迭代只允许发生在相邻阶段之间，那么任何非相邻的反馈都只能被当作异常处理，代价被记在项目后期。Royce 的整篇论文其实是在抵抗这种读法，他给出的五条建议全部服务于压缩回溯距离。

6.3 Royce 的五条建议

讲者把论文的这部分概括为一次“关键跳跃”：既然风险最终在测试处爆炸，而爆炸的代价由回溯距离决定，那么所有补救的本质就是同一件事——把“发现问题”的时间点提前，把回溯距离压短。五条建议都是从这一条推出来的，而不是凑数的最佳实践。

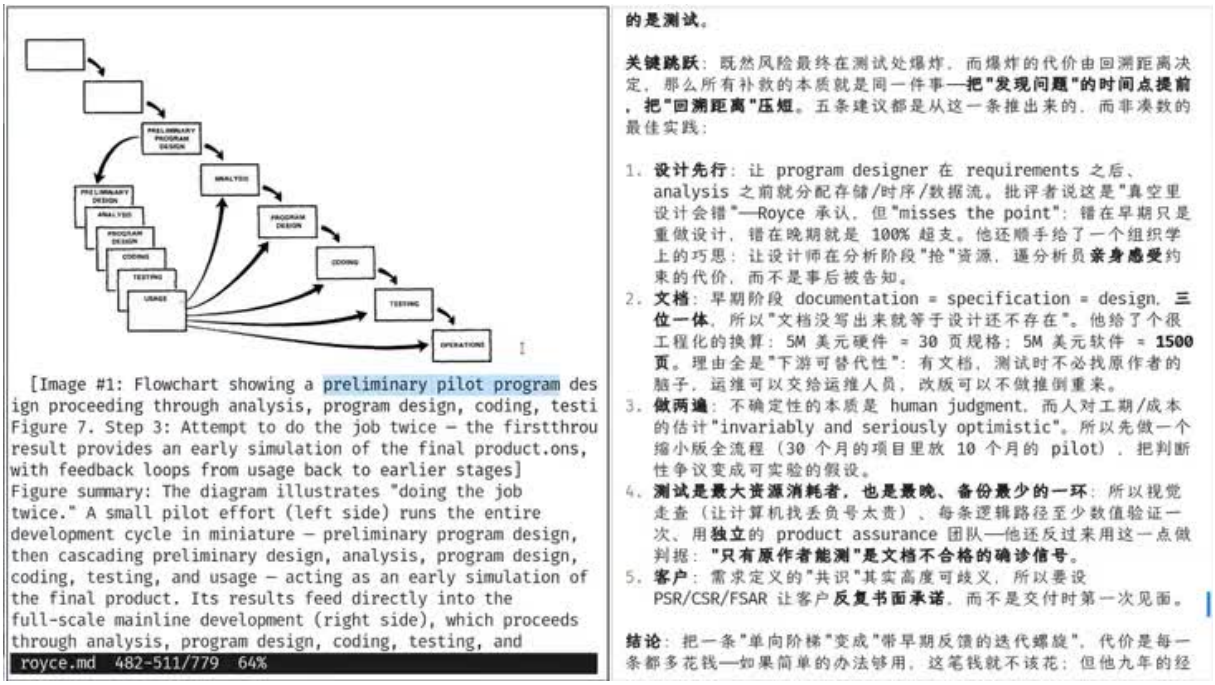


图 17: Royce 五条建议的总览页（讲者用 AI 阅读助手做的中文重述），以及 Figure 7 所描述的 do it twice 流程：pilot 全流程先跑一遍，再进入主线开发。[17]

第一条是设计先行：让 program designer 在需求分析之后、分析之前就分配存储、时序与数据流。批评者说这是在真空里做设计，必然出错；Royce 承认会错，但这是 miss the point——错在早期只是重做设计，错在晚期就是 100% 超支。他还顺手给了一个组织学上的巧思：让设计师在分析阶段“抢”资源，逼分析员亲身感受约束的代价，而不是事后被告知。

第二条是文档。在早期阶段，documentation 等于 specification 等于 design，三位一体，所以“文档没有写出来就等于设计还不存在”。他给了一个很工程化的换算：5M 美元的硬件对应 30 页规格，而 5M 美元的软件对应 1500 页规格。理由全部落在下游的可替代性上：有文档，测试时不必去找原作者，运维可以交给运维人员，改版可以不做推倒重来。

第三条是做两遍，也就是 do it twice。不确定性的本质是 human judgment，而人对工期与成本的估计 “invariably and seriously optimistic”，所以要先做一个缩小版的全流程：在一个 30 个月的项目里放一个 10 个月的 pilot，把判断性的争议变成可以用实验回答的假设。

[17] 视频画面时间区间：00:58:45–00:58:58。若为裁剪图，时间区间仍对应原视频画面。

第四条关于测试：测试是最大的资源消耗者，也是最晚、备份最少的一环。因此要用视觉走查代替昂贵的自动检查（让计算机去找一个丢掉的负号太贵）、每条逻辑路径至少做一次数值验证、用独立的 product assurance 团队。他还反过来用这一点做判据：“只有原作者能测”是文档不合格的确诊信号。

第五条关于客户：需求定义中的“共识”其实高度可歧义，所以要设 PSR、CSR、FSAR 这类文档，让客户反复书面承诺，而不是等到交付时第一次见面。讲者的结论是：Royce 把一条单向阶梯改成了带早期反馈的迭代螺旋，代价是每一条都多花钱；如果简单的办法够用，这笔钱就不该花。

表 1: Royce 五条建议及其压缩的回溯距离（本讲义据视频 00:58:45 前后的幻灯片与讲解整理）

建议	具体做法	被压缩的回溯距离
设计先行	需求之后、分析之前就分配存储、时序与数据流	让结构性错误在实现之前暴露，避免在测试处回推到需求基线
文档	documentation = specification = design	让下游能在不打扰原作者的前提下验证与维护，降低交接成本
做两遍	30 个月的项目里放 10 个月的 pilot	把工期与成本这类判断性争议变成可实验的假设
测试前置	视觉走查、每条逻辑路径至少数值验证一次、独立验收团队	测试是最大的资源消耗者，也是回溯距离最长的一环
客户承诺	PSR、CSR、FSAR 让客户反复书面确认	把“需求共识”的可歧义性在早期收敛

6.4 误读链：从教科书到采购制度

讲者用几页幻灯片复盘了“瀑布”这个名字是怎么被安到 Royce 头上的。第一层是术语反过来定义对象：waterfall 是批评者造的贬义标签，先有贬义，后被回溯挂到 Royce 头上。他给了一个类比：Big Bang 原本是 Hoyle 为挖苦而造的说法，结果成了宇宙学的正式名称；名字一旦流行，就自带一套“线性、不可回头”的语义，再没有人回去对原文。第二层是传播链：Sommerville、Pressman 的教科书被抄进全世界的考卷，中间没有人回原文。

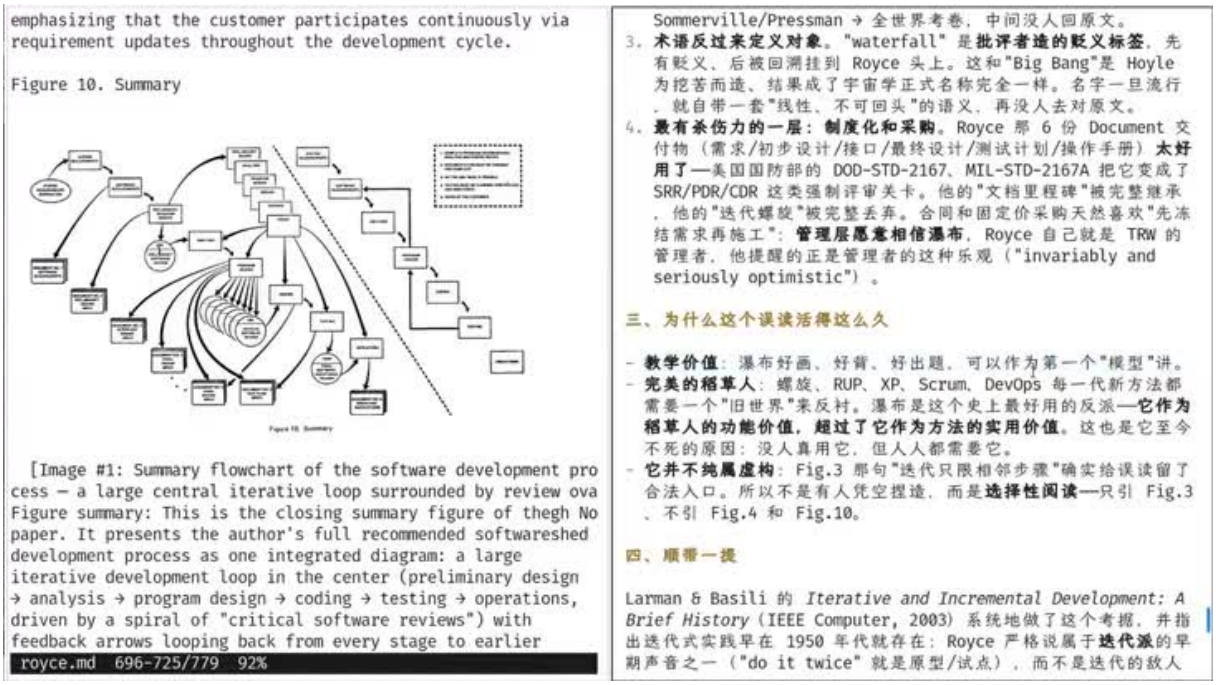


图 18: 误读链的复盘页：术语反定义、制度化与采购，以及论文 Figure 10 的总结图；幻灯片指出 Royce 的“文档里程碑”被继承，“迭代螺旋”被丢弃。[18]

最有杀伤力的一层是制度化与采购。Royce 卖出的那 6 份 Document 交付物（需求、初步设计、接口、最终设计、测试计划、操作手册）太好用了，美国国防部的 DOD-STD-2167 与 MIL-STD-2167A 把它变成了 SRR、PDR、CDR 这类强制评审关卡：他的“文档里程碑”被完整继承，“迭代螺旋”被完整丢弃。合同与固定价采购天然喜欢“先冻结需求再施工”，而管理层也愿意相信瀑布；讲者提醒，Royce 自己就是 TRW 的管理者，他提醒的正是管理者这种“invariably and seriously optimistic”的乐观。

幻灯片还补了一条考据：Larman 与 Basili 在《Iterative and Incremental Development: A Brief History》（IEEE Computer，2003）里系统整理了这段历史，指出迭代式实践在 1950 年代就已经存在，Royce 严格说属于迭代派的早期声音之一，do it twice 就是原型与试点，而不是迭代的敌人。

6.5 为什么这个误读活得这么久

讲者给出了四个原因，它们彼此并不冲突。第一是教学价值：瀑布好画、好背、好出题，可以作为第一个“模型”来讲。第二是它是一个完美的稻草人：螺旋、RUP、XP、Scrum、DevOps，每一代新方法都需要一个“旧世界”来反衬，瀑布是这个史上最好用的反派。第三是它并不纯属虚构：论文 Figure 3 那句“迭代只限相邻步骤”确实给误读留了合法入口，所以不是有人凭空捏造，而是选择性阅读——只引 Figure 3，不引 Figure 4 和 Figure 10。第四是它的功能价值超过了实用价值：没人真用它，但人人都需要它。

讲者由此延伸出一个更普遍的判断：经典之所以值得读，是因为它记录了一个领域最初遇到的最根本问题，以及人们怎样找到出路；一旦经过多层消化，你得到的东西可能已经不是原来那个东西。读原文在今天反而更容易理解，因为你可以直接代入历史背景，也可以借助 AI 工具把原文拆开。

[18] 视频画面时间区间：01:03:58–01:04:23。若为裁剪图，时间区间仍对应原视频画面。

6.6 本章小结

本节回答“Royce 究竟写了什么、为什么被读反”。Royce 的判据是：缺陷被发现得越晚，返工代价越大，而代价由回溯距离决定；他的五条建议全部服务于把发现问题的时间点提前、把回溯距离压短；do it twice 是缩小版的全流程，用于把判断性争议变成可实验的假设。误读之所以扩散，是因为“瀑布”这个标签自带贬义语义，教科书与考卷把它固化成常识，采购制度又把文档里程碑变成强制关卡。理解这一段历史之后，下一节就可以用一条更本质的判据把 Royce、MVP、敏捷与 TDD 放在同一个坐标系里。

7 用数据依赖图重新理解瀑布、MVP 与 TDD

这一节是讲者对整段历史的重构。他先补上 Royce 全集里最锋利的观察，再给出自己的模型——把开发过程看成一张数据依赖图——然后用同一个模型解释几种看似对立的方法。

7.1 Testing：第一次“被经历”而非“被分析”

讲者称这是全文最锋利的观察：Testing 是 timing、storage、I/O 第一次“被经历”，而不是“被分析”的地方。这些系统行为很难在实际运行之前被精确分析，如果实际运行不满足约束，局部修补代码往往不够；代价会沿着依赖一路向上：不满足约束、要大改设计、推翻需求基线、一切重来，成本与进度可能翻倍。



图 19: Testing 是 timing/storage/I/O 第一次“被经历”的地方；幻灯片用“算法正确但赶不上控制周期”说明为什么算法正确也救不了系统设计。[19]

幻灯片给了一个很尖锐的例子：算法正确算出结果，却赶不上控制周期；在这种情况下，算法层面的正确无法挽救系统设计的错误。讲者由此说明为什么“把测试提前”不是赶工技巧，而是唯一能改变代价结构的动作：既然这些性质只能在运行中被经历，那么越早经历，返工的距离越短。

[19] 视频画面时间区间：01:07:07–01:07:25。若为裁剪图，时间区间仍对应原视频画面。

7.2 把瀑布模型看成一张数据依赖图

接下来是本节的核心。讲者的做法是把瀑布模型的每个阶段想象成一个 program。人是神经网络，也会做 chain of thought；他在思考时仍然会对齐到语言，哪怕不出声，也会用“我要这样，会有什么后果”这样的句子对齐脑内的思维链。因此“人写产物”与“程序产生产物”可以用同一个结构来描述：需求里的若干个点、设计里的点、实现里的点之间有关联，而关联会传递。于是整个开发过程就是一张数据依赖图，一个带方向的、无环的图。



图 20: “瀑布模型就是 Data Dependency Graph”：阶段之间有依赖，因此改动会沿依赖传播；幻灯片同时给出 TDD 的推论。[20]

这张图直接把上一节的问题解释清楚了：一旦测试错了，一路往上追溯，发现几个设计点错了，所有依赖它们的东西全部作废，这就是成本高的来源。讲者承认这是他个人的理解（他是做操作系统的，习惯用操作系统里习得的概念解释世界），但他认为这个理解自洽，而且“基本就是软件工程后来真正采用的形式化方式”。

最重要的推论是：如果依赖是代价的来源，那么不要一次性地产生大量依赖。宽度的构建方式是先把第一层全部做完再做第二层，一旦上游出错，整层都废；更好的做法是深度优先：先做优先级最高的那一条依赖链路，走通之后再向右扩展。幻灯片把这条推论说得更直接：不做需求（Intent/Spec），设计就会漂移；设计（Spec）错了，实现就会漂移；没有实现，谁会想去测试？但产物之间有依赖，不等于所有阶段只能完整地做一遍。这正是 TDD 的入口：拿 testcase 当需求与设计，写下预期行为只需要需求，验证实现是否满足预期才需要实现。

[20] 视频画面时间区间：01:10:40–01:11:34。若为裁剪图，时间区间仍对应原视频画面。

一条可迁移的判据：依赖范围 × 反馈距离

讲者把 Royce 的一切说法压缩成两个量：一次改动会影响多少依赖它的产物（依赖范围），以及多久才能得到“它到底对不对”的反馈（反馈距离）。MVP、敏捷、TDD 都在做同一件事：在不减少必要依赖的前提下，把反馈距离压短；它们之间的差别主要在于时间尺度，而不在于是否承认依赖存在。

7.3 Do It Twice、MVP、敏捷与 TDD 的关系

Royce 的 do it twice 与今天的 MVP 是同一件事的两种叫法，但侧重点不同。MVP 是 minimum viable product：先做一遍，从中学习，再完善产品；Royce 的 do it twice 是先用 pilot model 经历关键问题，重点验证技术与运行风险，而 MVP 更关注用最小投入验证用户与业务假设。两者共同的地方是：让关键假设尽早遇到证据。



图 21: Do It Twice 与 MVP 的对照页：Pilot 验证技术与运行风险，MVP 验证用户与业务假设；幻灯片进一步用敏捷解释“反复做 pilot”。[21]

用敏捷来理解，就是不必把 pilot 单独隔成一个大阶段，而是把试做、反馈、修正反复快速地做。幻灯片给出的对比更完整：瀑布是晚验证、长返工；MVP 与 pilot 是先验证高风险假设，再修订需求与设计；敏捷则是小批量、短反馈，用一个个小迭代把验证密度提上来。

[21] 视频画面时间区间：01:07:34–01:09:10。若为裁剪图，时间区间仍对应原视频画面。

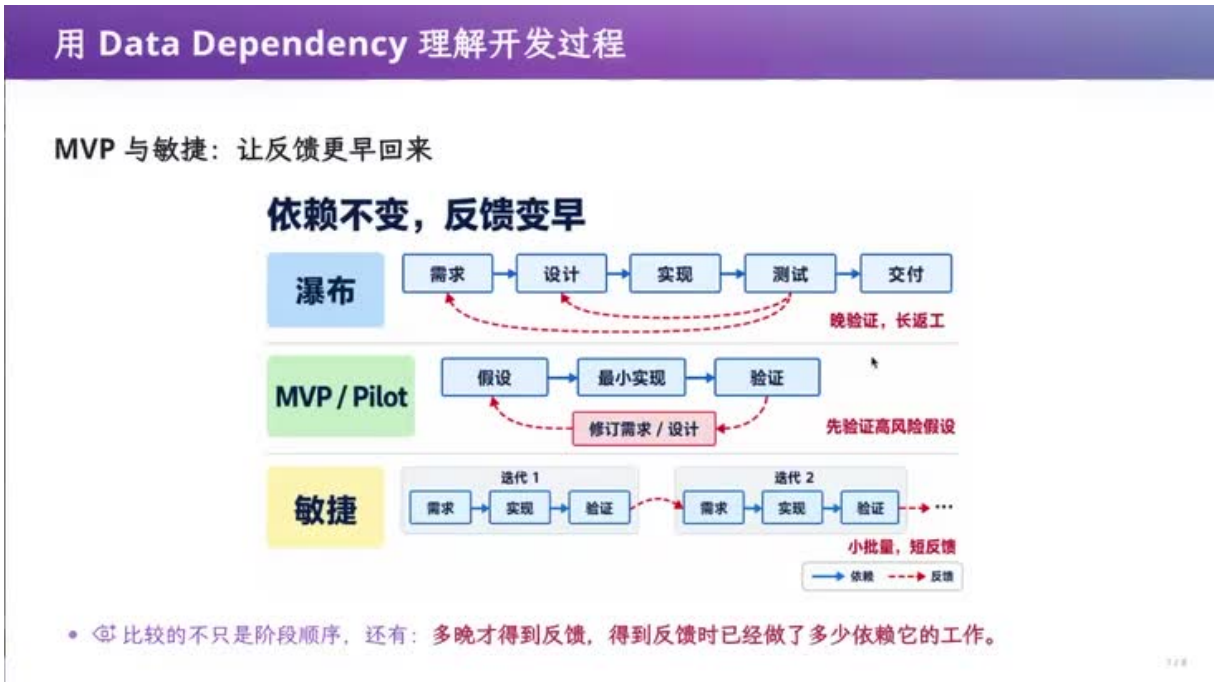


图 22: 用数据依赖理解开发过程：瀑布、MVP/Pilot 与敏捷三种过程在“反馈回来的早晚”上的差别，以及它们共同保留的依赖结构。[22]

表 2: 三种过程的对比（本讲义据视频 01:13:28 前后的幻灯片整理）

过程	验证时点	一次改动的依赖范围	返工距离
瀑布	全部实现完成后的测试阶段	一次改动可能覆盖整层产物	长，可能回溯到需求基线
MVP 与 Pilot	先做缩小版全流程	只覆盖被验证的关键假设	短，但会重做部分设计
敏捷	每个小迭代结束即验证	每次只覆盖一个小批量	最短，反馈密度最高

讲者因此认为，这些新方法并没有颠覆旧经验：依赖关系不变，变的是反馈回来的早晚。他特别强调，比较的不只是阶段顺序，还有“多晚才得到反馈、得到反馈时已经做了多少依赖它的工作”——这句话把 MVP、敏捷、TDD 与 Royce 的判据统一到了一条轴上。

7.4 阅读经典与一手信息

讲者在这一节插入了自己的研究方法。他认为经典记录了一个领域最初遇到、通常也是最根本的问题，以及它如何在方向上找到出路；在这些经典之上会长出后来世界的许多分支，甚至可能发现它一开始把全人类带歪了方向（他举了直流电与交流电的例子）。带着今天的经验回到当年的问题，往往会变得非常容易理解。

他也解释了为什么今天才有可能这样做。过去要查证一个知识网络几乎不可能：图书馆索引不好用，找一份精确资料要花掉无穷多的时间，所以只能依赖教科书做有损压缩；好的老师能共情学生，知道学生哪里会不明白，因此能把一手信息压缩得损失较小。现在每个人都可以直接读到一手信息，讲者用 AI 阅读助手把 Royce 的论文处理成了 royce.md，把插图直接嵌进 markdown，还能在命令行里分页阅读与检索。由此他的态度是：自己把知识拆成一个个“发现的点”，再把每个点与它的原文拿到一起，做一份属于自己的压缩。

[22] 视频画面时间区间：01:13:28–01:14:17。若为裁剪图，时间区间仍对应原视频画面。

讲者最后回到人的位置：AI 也许有一天会学会 taste，但从论文里学品味与从实践中学品味是两件事；人类现在还有一点傲慢，希望能理解 AI 做了什么、判断 AI 做得对不对，也许当 AI 永远正确的那一天，人就会放弃理解；至少在今天，能驾驭、能理解 AI 的人会活下来。

7.5 本章小结

本节用一条判据把四种方法统一起来：开发过程是一张数据依赖图，修改的代价沿依赖传播，因此压缩反馈距离就是压缩代价；Royce 的 do it twice、MVP、敏捷与 TDD 都是这条判据在不同时间尺度上的实现。同时，这一节也解释了为什么经典值得读：原文里保留着问题最初的形状，而今天的工具第一次让普通人可以直接对照原文。下一节讨论软件工程在几十年里为此发明了哪些方法与工具。

8 软件工程的方法和工具：为每个阶段造一门语言

讲者对软件工程的运作定义非常直接：为每一个阶段创建方法学和工具，尽可能减少返工的概率。本节顺着这个定义，看这些方法在需求、设计、测试与维护四个位置分别长成了什么样子。

8.1 不同阶段，不同的“语言”

在没有大语言模型这种“万能编译器”的时代，不同阶段需要不同的语言来承载信息。幻灯片把三个阶段的语言列了出来：需求分析用文档、用例与界面原型，回答“我们到底想要什么”；系统设计用实体关系、状态迁移、模块与接口，回答“系统怎样组织起来”；代码实现则使用编程语言，而计算机科学的大部分课程都集中在这里，这也是一种自下而上的学习方法。

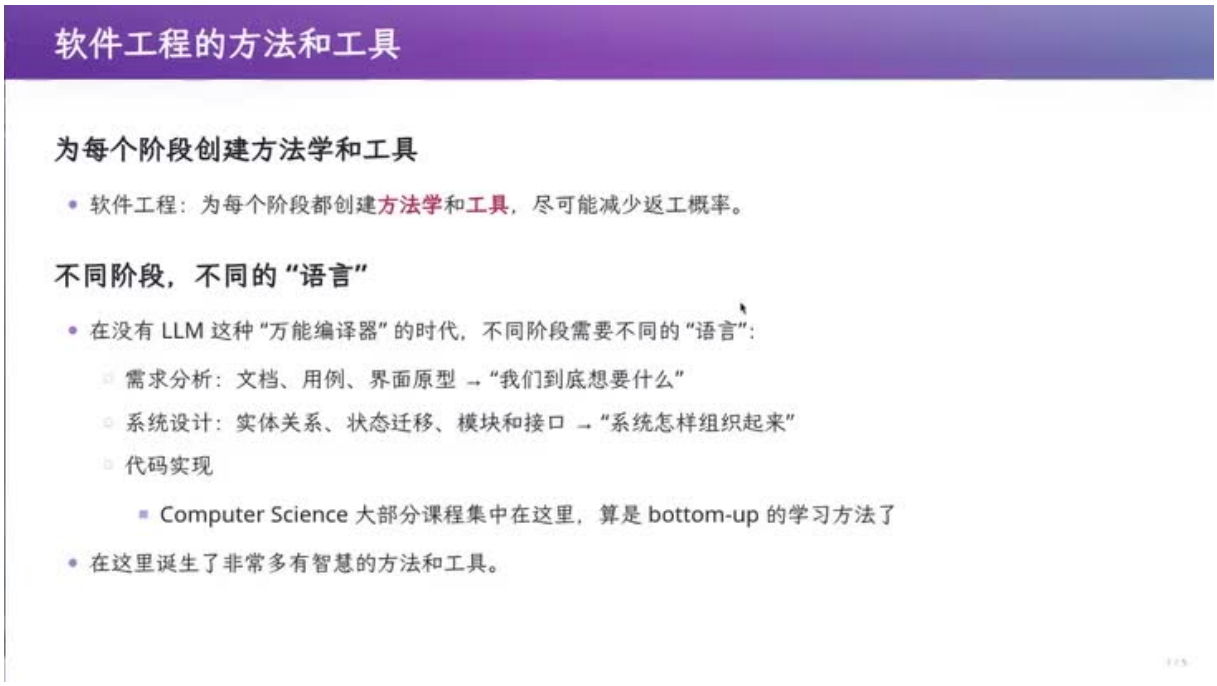


图 23: 每个阶段使用不同的“语言”：需求分析、系统设计与代码实现各自的形式化程度不同，而形式化程度沿阶段递增。[23]

[23] 视频画面时间区间：01:18:51–01:23:06。若为裁剪图，时间区间仍对应原视频画面。

讲者在这里指出了一个 gap：从自然语言到编程语言之间，过去没有一种语言能直接承担翻译。文档最大的缺点是写文档的是人：人会产生幻觉、会产生 typo，两个人讲同一件事时无法对齐——你和室友之间已经发生过无数次，你说的是这个，他理解成了那个。如果设计师想要为“减少返工概率”做点实事，他要造的就是一门介于自然语言与编程语言之间的设计语言，也就是一种领域特定语言。

表 3: 阶段与语言的形式化程度（本讲义据视频 01:18:51 前后的幻灯片整理）

阶段	使用的语言	形式化程度	它要防住的失败
需求分析	文档、用例、界面原型	低，自然语言为主	意图没写清楚，或各方理解不一致
系统设计	实体关系、状态迁移、模块与接口、checklist	中，半形式化	设计漂移，或没有留下可检查的结构
代码实现	编程语言	高，机器可执行	实现与规格不一致（可由编译与测试暴露）

8.2 checklist 为什么不是 nonsense

讲者在这里讲了一段很容易被学生忽略的内容。他说，软件学院的学生上软件工程课时最讨厌 checklist：一群理科生被要求只有当所有勾都打上才能进入下一个阶段。但从“形式化程度递增”的角度看，checklist 其实是一门中间语言：它有格式规范（必须一条一条，每条必须打勾或打叉），因此有自己的语义；虽然语义由人来检查，但人对勾和叉的对齐比人对一段自由文档的对齐更正式。这就说明了软件工程的一个规律：从设计阶段往下，确定性语义的程度越来越高，正好对应 Intent 到 Spec 到 Impl 之间 gap 的逐步收敛。

两个具体动作：两两对照与客户确认

checklist 里有两个动作看起来最没意思，却最有效。一个是要求把所有文档两两摆在一起审查一遍：这个动作不产生任何代码，但能避免不同文档之间的 gap 发生漂移，讲者说这都是从失败里换来的智慧。另一个是“是否与客户确认过”这一勾：勾上之后，Intent 不符合客户要求的概率会大幅下降。这两个动作都是人文的、没有代码的工具。

8.3 需求与测试：把规则变得可检查

到了需求与测试这一层，讲者给了两个具体的工具。

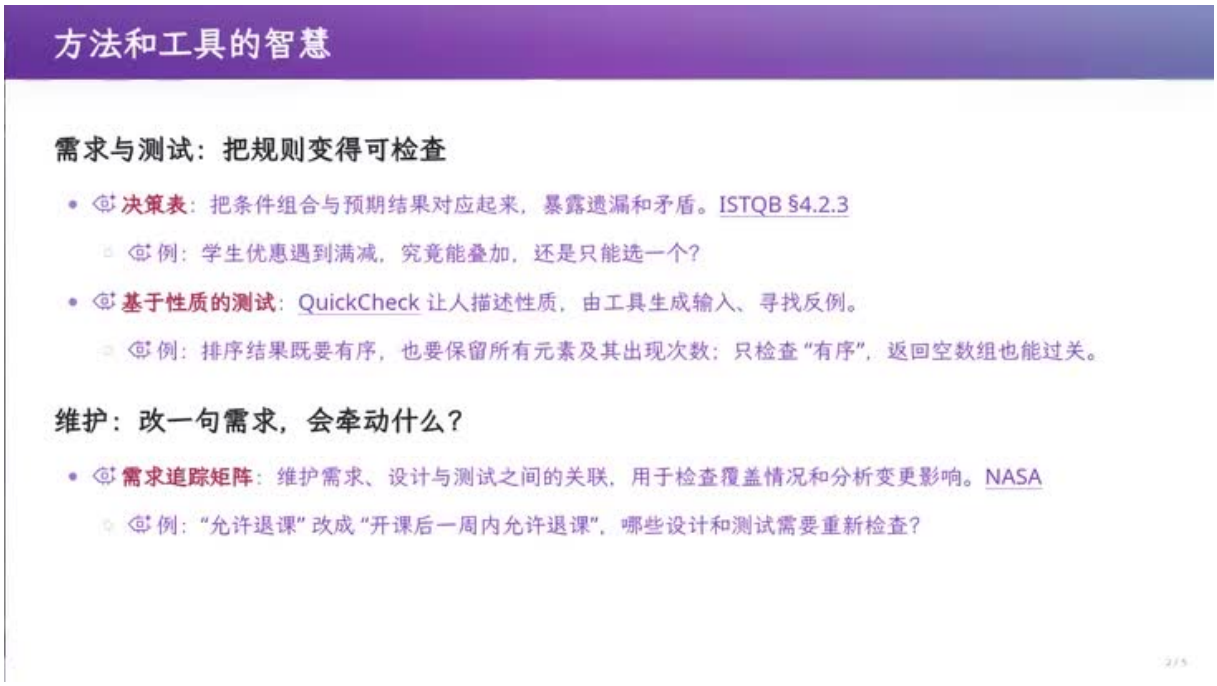


图 24: 方法与工具的智慧：决策表把条件组合与预期结果对应起来，性质测试（QuickCheck）让人描述性质、由工具生成输入寻找反例；下半页是需求追踪矩阵。[24]

第一个工具是决策表：把条件的组合与预期结果对应起来，从而暴露遗漏与矛盾（幻灯片标注了 ISTQB 54.2.3 这个出处）。它举的例子是“学生优惠遇到满减，究竟能叠加，还是只能选一个”。这种问题的特点是：需求文档里很难自然地写到它，不写就默认不会发生，直到测试或线上才被发现；决策表把条件空间展开，迫使你把每一格都填上。

第二个工具是性质测试：QuickCheck 让人描述性质，由工具生成输入、寻找反例。幻灯片给了排序的例子：排序结果既要有序，也要保留所有元素及其出现次数；如果只检查“有序”，那么返回空数组也能过关。这个例子说明了性质测试的价值与陷阱——性质写得不好，测试就会给出虚假的安心。

讲者由此解释了这一层背后的逻辑：任何从上到下的翻译都会出错，从规格翻译到代码尤其容易漏掉隐含的规格，比如程序不能崩、不能有空指针、资源不能泄漏。你不能断言自己写对了，但可以把性质写下来，让它自动变成成千上万个测试用例；如果全部通过，你对代码的信心就会大一些。除非你最终写出机器检查的证明，那种信心会更大。

8.4 维护：改一句需求会牵动什么

第三个工具属于维护阶段：需求追踪矩阵。幻灯片引用了 NASA 的做法，它维护需求、设计与测试之间的关联，用于检查覆盖情况并分析变更影响。它给的例子是：“允许退课”改成“开课后一周内允许退课”，有哪些设计和测试需要重新检查？

讲者在这里把之前的数据依赖图接了回来：traceability 做的就是把这套依赖关系显式化。他还解释了文件系统视角的由来：从文件系统的角度看，软件就是很多文件，例如 docs 目录下的若干 markdown、source 目录下的若干源文件，再加上一个 README；这是“因为大家一开始用了文件系统而不得不做的 work around”。而追溯性要求每一个代码与每一个需求点都能对上、需求和需求

[24] 视频画面时间区间：01:24:15–01:27:30。若为裁剪图，时间区间仍对应原视频画面。

之间能对上、需求和设计之间能对上；有了这种独立的互相交叉检验，你对软件质量的信心就会更大。

8.5 谁来完成这些工作

本节末尾，讲者把这个话题拉回到学生的职业预期上。学生时代的假设是：只要把自己变成世界上最厉害的码农，就能在软件企业找到很好的工作，这是社会给的 contract。但从资本或任务的角度看，没有一个码农能单独完成项目，管理者只能问“码农有好有坏、我要花多少钱雇多少人”。而在今天，你手上可能马上就有 2500 个并发的 DeepSeek，它“绝对算是相当资深的码农”。于是每个人都得学着当产品经理，第一门课就学做产品，因为你面对的问题已经变成：手里有一批足够强的执行者，你如何组织、如何定义验收。讲者说，这个 mindset 要改一改。

8.6 本章小结

本节回答了“软件工程几十年里究竟造出了什么”。它为每个阶段造了一门中间语言：需求阶段的用例与原型、设计阶段的实体关系与 checklist、实现阶段的编程语言，形式化程度沿阶段递增，恰好对应 Intent 到 Spec 到 Impl 的 gap；需求与测试阶段的决策表、性质测试、需求追踪矩阵，以及维护阶段的追溯性，都是同一件事的不同形态——把隐含的规则与依赖变成可检查的对象。下一节把这件事做到极致：把规格直接写进编程语言。

9 把契约写进语言：从 Eiffel 到 Dafny 与 Verus

讲者把这一节称为“软工人在 spec 上横跳”的最激进一步：不再满足于用文档描述规格，而是让编程语言本身能表达规格，并且让工具去检查它。

9.1 Design by Contract：调用者与实现者各自负责什么

契约编程来自 Bertrand Meyer 的 Eiffel。它的直觉很像人类合作中的契约：接口写清双方的义务与权利。幻灯片给了三个组成部分：前置条件由调用者保证，即调用前必须满足的使用条件；后置条件在前置条件成立时由实现者保证，即正常返回后的结果；类不变量在对象对外可用时必须成立，各公开操作都要维护它。



图 25: Design by Contract：前置条件、后置条件与类不变量，以及“不允许透支的账户取款”这个具体例子。[25]

幻灯片用账户取款把三者写成了可以检查的式子。调用 `withdraw(amount)` 之前，调用者必须保证余额足够且金额合法：

$$0 < amount < balance$$

正常返回之后，实现者必须保证余额恰好减少了这次取走的金额：

$$balance = balance_{\text{old}} - amount$$

并且在整个对象对外可用期间，类不变量都必须成立：

$$balance \geq 0$$

其中 $amount$ 是本次取款金额，$balance$ 是调用前的余额，$balance_{\text{old}}$ 是用于描述后置条件的调用前余额快照。三条式子分别定义了调用者的义务、实现者的义务，以及对象在任何公开操作前后都必须保持的性质。

这个例子真正有价值的地方，是它迫使你做业务选择。如果“余额不足”是正常结果而不是错误，那么“返回失败、余额不变”就应该写进接口约定；如果你默认这件事永远不会发生，那么第一个 MVP 里很可能就是让它 crash，或者把它排除在规格空间之外。讲者说，做第一个 MVP 时你一定不会管这些事；但当软件不断向后演进时，这份 contract 就变得非常重要。

[25] 视频画面时间区间：01:28:04–01:28:24。若为裁剪图，时间区间仍对应原视频画面。

MVP 阶段最容易漏掉的正是这些“不会发生”的情况

Royce 的 pilot、今天的 MVP，目标都是把关键假设尽早变成证据；而契约编程提醒的是另一类假设：那些被默默排除在规格之外的边界情况，例如余额不足、并发冲突、硬件失效。它们在 MVP 阶段通常被忽略，但在长期运行的软件里正是返工与事故的源头。

9.2 Eiffel：把契约写进语法

Eiffel 的做法是把这三件事变成语言的一部分。幻灯片给了一段“上古语法”：



图 26: 把契约写进语言：Eiffel 的 require / do / ensure 结构，以及 Dafny 与 Verus 这条延续至今的路线。[26]

```
function(args) is
        -- Comment
    require
        Precondition
    do
        Routine
    ensure
        Postcondition
    end
```

Listing 1: Eiffel 中一个带契约的例程骨架（对应视频 01:31:06 的幻灯片）

讲者逐行解释了这段骨架：先写函数是干什么的文档，再用 require 写前置条件，do 后面是实现，ensure 写完成后能保证什么。他强调，Eiffel 鼓励你先不写 do，把各个模块的前置条件与后置条件写出来——因为在写 precondition 与 postcondition 的过程中，你会自然地去关联自己的设计，

[26] 视频画面时间区间：01:31:06–01:32:22。若为裁剪图，时间区间仍对应原视频画面。

例如“余额不足到底应该怎么处理”。契约既是接口文档，也能在开启断言检查时检验本次执行，这正好对应了软件工程的目标：让规格成为可以被机器检查的对象。

讲者顺便交代了时间线：Eiffel 这个语言大约出现在 1986 年。他还指出，软工人这几十年的努力就是在 spec 这一层横跳：从自然语言，到半自然的语言，到能接纳逻辑公式的语言（它可以没有代码，单纯检验逻辑，例如检验函数调用前后的逻辑能不能成立），再到今天的编程语言本体。

9.3 今天的活跃方向与一个可检验的预测

幻灯片把这条线的今天写在了同一页：Dafny 内置了规约与静态验证器，Verus 为 Rust 的一个子集提供静态验证；它们进一步尝试证明：在给定假设下，实现满足写出的规约。讲者把它称为大家共同的梦想：写一个程序，可以自动证明“前置条件满足时后置条件必然满足”，无论实现怎么写都能证明；而且还要证明整个系统把所有前后置条件连起来之后，能够满足最上层的 intent。

讲者在这里给了一个可以对照历史检验的预测。他说，在 ChatGPT 出现之前，他在操作系统课上就预言过：三十年之后（按照今天的进展看，也许十到十五年），如果一个程序里的 assert 无法被证明，那么程序就不能通过编译——也就是说，你写的断言与业务逻辑都必须可证明，property-based testing 也许就不再需要了。这个预测的注意点在于，它把“证明”从少数关键系统推向普通开发者的日常工作，而这恰好也是软件工程一直在做的事：把模糊的意图翻译成机器能检查的形式。

讲者最后提醒，AI 让这件事的边界发生了变化：自然语言“对不对齐”的问题被解决了，checklist“不好检查”的问题也被解决了，剩下的只是那个 gap——我没有想清楚，我没有说清楚。一旦想清楚、说清楚，剩下的部分可能真的都能自动化。

9.4 本章小结

本节回答的是“规格能不能变成程序的一部分”。契约编程把接口的义务拆成前置条件、后置条件与类不变量，Eiffel 把它们写进了语法，Dafny 与 Verus 把这条线推进到静态验证；其共同目标是让“实现满足规约”成为可证明、可检查的事实。讲者的预测（断言不可证明就不能编译，大约十到十五年）是这条路线的一个自然终点，也把问题重新交还给开发者：能被证明的部分取决于你能不能把意图说清楚。下一节讨论这条路线在“没有代码可用时”的极端形态。

10 UML：用语言的方法解决 Intent–Spec–Impl Gap

讲座的最后一个案例是 UML。它的野心比契约编程更大：不是为单个函数写规格，而是想给整个面向对象的世界提供一种中间语言，让需求和设计阶段也能“对齐”。

10.1 为什么需要中间语言

讲者的解释从语言的性质开始。人类用自然语言沟通，自然语言直连大脑的 latent space：我说“我今天看到一个美女”，每个人在自己的 latent space 里都构造了一个美女，但她们都不一样；如果要求大家用 Blender 把这个形象建模出来，差异会更明显。所以自然语言很难对齐。数学语言不一样，它是全世界自然对齐的，因为大家知道什么是自然数，而它是从很少的公理一点点构造出来的；一旦对齐完成，用皮亚诺公理就能把整个计算机世界构建起来，计算机世界里的同一个东西，其语义是绝对对齐的。UML 想要做的，就是尽可能对齐自然语言世界里的需求与设计：它通过定义模型元素与半形式化的语义，把能对齐的部分与不能对齐的部分剥离出来。



图 27: UML 的目标与历史：它汇合了 Booch、OMT、OOSE 等方法，用统一的记号与语义减少“没有代码时”的语义漂移。[27]

幻灯片给了两个能对齐的例子。类图的语义是确定的：一个 Student 类里有一个 name 字段，它就能变成源代码里的一个东西。顺序图的语义也是确定的：它规定了从前到后的执行顺序，因此不需要用自然语言去描述并发时“谁先谁后”这种有歧义的问题。UML 的历史背景是它汇合了 Booch、OMT、OOSE 等方法，由 OMG 标准化，让团队与工具能使用共同的建模语言；用讲者的话说，它要在没有代码的时候最大程度减少语义漂移。

自然语言、数学语言与中间语言各自解决什么

自然语言的优点是可以直连人的内部表示，缺点是不同人的表示不同；数学语言的优点是对齐绝对确定，缺点是表达范围窄、写起来慢；UML 这类中间语言想插在两者中间，把“能对齐的部分”变成有语义的模型元素，把“必须留白的部分”交还给自然语言。讲者说，这个思路在今天依然值得学，哪怕它的产品形态已经过时。

10.2 阅读 796 页的规范

UML 规范的体量是这一节的一个具体数字：796 页。讲者的处理方式本身就是一个 AI 时代的样本。他先在课上讲了 Computer Science 的 bottom-up 学习经验：上大学时学过 UML 但感觉没学懂，甚至怀疑老师可能也不太懂；PhD 时也读了一些论文，印象依然是模糊的。这次他换了方式：把规范 dump 下来，切成带 overlap 的块，分给若干 sub agent，每个 agent 读五十页，把每块里的软件工程智慧提取出来，再对照自己的 lecture notes 总结成可用的材料。他提到这次阅读使用了 Kimi 赠送的 $ 699 套餐，折算下来大约花了 $ 5.3，占套餐五小时额度的 18%。

[27] 视频画面时间区间：01:32:34–01:35:47。若为裁剪图，时间区间仍对应原视频画面。



图 28: 讲座最后一页：796 页规范从头到尾只在回答一个问题——哪些意图必须显式钉死、哪些可以留白；稳定性解决自己会不会变，显式表达解决别人能不能知道。[28]

讲者用这个数字提出了一个社会性的问题：如果学习要靠这样的上下文消耗，那么拥有好模型与没有好模型的学生之间的学习效率与成本会拉得很开。大项目很烧 token，因为要在大上下文里载入很多东西，100 万上下文很快就会被填满；而学习流程本来不需要把整份规范塞进去，可以用压缩过的版本。他问的问题是：当所有人都要求教育平权时，我们没有那么多算力与资源，这件事怎么办？他没有给答案，把它留成了开放问题。

10.3 判断与边界：UML 死了，问题没有

讲者对 UML 的最终判断很直接：在 LLM 时代，UML 作为产品已经死了。它的全部假设是“自然语言和形式语言无法对齐，所以需要一层一层的中间语言”；而今天自然语言可以与任何东西对齐，有问题直接问就可以。但他同时强调，UML 设计时的 first principle 一直在指导我们：796 页规范从头到尾只在反复回答一个问题——哪些意图必须显式钉死，哪些可以留白；它的全部设计都是对这个问题的回答。

幻灯片最后一页把这个判断写得比讲者的口头表达更精确。如果你是一个从不“朝令夕改”的 deterministic people，你其实不需要 UML，因为你的行为总是可重复的；但如果你要和别人对齐，或者要和下一时刻的自己对齐，语义对齐就很重要。它给出的分界是：稳定性解决自己会不会变，显式表达解决别人能不能知道。这两句话正好总结了整场讲座：软件工程史上发明的一切工具，都在解决“把意图显式表达出来”这件事；而 LLM 改变的只是表达与检查的成本，没有取消这个问题。

10.4 本章小结

本节回答了“当自然语言本身也能被机器理解时，中间语言还剩什么”。UML 的目标是减少没有代码时的语义漂移，随着 LLM 使自然语言与形式语言直接互通，它作为产品已经过时；但它留

[28] 视频画面时间区间：01:39:27–01:39:44。若为裁剪图，时间区间仍对应原视频画面。

下的问题仍然成立：哪些意图必须显式钉死、哪些可以留白，以及如何与别人、与未来的自己对齐。这个问题就是我们今天用 agent 写代码时最需要回答的问题。

11 总结与延伸

11.1 讲者的收束

讲座最后没有额外的总结段落，讲者讲完 UML 这一页就直接下课了。但他在全课中反复给过几句可以作为结论的话：软件工程的智慧没有消失；能驾驭、能理解 AI 的人会活下来；剩下的问题只是“我没有想清楚，我没有说清楚”。这三句话分别对应立场、能力与任务。

11.2 本讲义对全课主线的压缩

把这 100 分钟的内容压缩成一条链条，大致是这样的：

一条主线：把意图变成可检查的对象

软件是 Intent、Spec、Impl 的组合体，三者之间的 gap 制造了软件危机；软件工程把这门手艺变成“为每个阶段造方法学和工具”，而这些工具的本质是按阶段提高形式化程度，让意图中越来越多的部分变成可检查的对象。Royce 给出的是这条路线最早的判据：缺陷的代价由回溯距离决定，所有补救都是把发现问题的时点提前。把开发过程看成数据依赖图之后，这条判据就变成了“压缩反馈距离”，而 MVP、敏捷、TDD、契约编程、决策表、性质测试、需求追踪矩阵都是它在不同尺度与不同阶段的实现。

在这个框架下，LLM 改变的是两件事的成本：自然语言与任意形式语言之间的翻译成本，以及检查一份规格是否被满足的成本。前者让 UML 这类中间语言失去产品价值，后者让 checklist 这类人文工具的执行变得廉价。没有改变的是：你必须先知道要什么（Intent），再决定怎样组织（Spec），然后才能判断实现是否合格（Impl）；而“哪些意图必须显式钉死、哪些可以留白”这个问题，在人机协作中只会变得更频繁，因为你要对齐的不再只是同事与未来的自己，还有若干个 agent。

11.3 边界与开放问题

这一讲里有几处内容是讲者的判断与预测，读者需要把它们与已经发生的事实区分开。第一，数据分布工程与 RSI 的飞轮、以及“每建一个实验室能力就上一个台阶”，是讲者基于公开新闻的推论，视频里没有给出训练细节或数字证据。第二，“软件公司会彻底消亡”“豆包手机与这类决策模型非常配”是讲者在产业层面的判断，他本人也在幻灯片上把这类内容表述为展望。第三，关于断言不可证明就不能编译的预测，讲者自己给出了十到十五年的区间，尚未验证。第四，把 UML 判为“产品已死”同样是一个判断，其依据是自然语言与形式语言的对齐成本下降，而不是对 UML 规范本身的技术评价。第五，教育平权与算力分配的问题是讲者主动留白的开放问题，本讲义不替它给出结论。

11.4 给你的三个可执行动作

如果要把这一讲用起来，可以落成三个动作。第一，用依赖范围与反馈距离审视手上的项目：列出关键产物之间的依赖，找出哪些假设最晚才会被验证，然后把它提前到第一个可运行的版本里。第二，把“不会发生的边界”写下来：余额不足、并发冲突、硬件失效、异常输入，它们是最容易在

MVP 中被默默排除、又最容易在长期运行时爆炸的部分。第三，回到一手材料：这一讲涉及的所有经典都可以直接读，带着今天的实践经验去读 Royce 的原文、Dijkstra 的两篇文章与 UML 规范，会比读教科书得到的东西更接近它本来的样子。

11.5 下一步要问的问题

讲者在课里埋了一个没有回答的问题：在“任务构造加结果验证”可以把越来越多智力活动变成训练对象的时代，我们还能剩下什么。对读者而言，一个可检验的版本是：在你所处的领域里，哪些判断目前还没有可验收的结果？如果你能把它变成一个可验收的结果，那么你既为模型准备了训练信号，也为自己保留了判断的位置。